```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P02.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S02_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   → S02_Main    T001④
    ///      Boot            UI T001b  Boot      =
    /// U185 19 § F         ——BootServices     LaunchComplianceFace
    ///          +           cfg_i18n   ⑦ armed
    ///       (3s)         ——              sim
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        bool _switchArmed;
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var face = LaunchComplianceFace.Build();
            float shownAt = Time.realtimeSinceStartup;
            var sequence = new BootSequence(() =&gt; { _switchArmed = true; return true; });
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            while (!(_switchArmed &amp;&amp; (face.EnterTapped
                || Time.realtimeSinceStartup - shownAt &gt;= LaunchComplianceFace.MinDisplaySeconds)))
                yield return null;
            yield return LoadMainRoutine();
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
```

```
using Sy.Platform;
namespace Sy.P02.Boot
{
    /// &lt;summary&gt;
    /// Boot        T001
    /// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦ S02     [BOOT] n/7
    ///  ⑦         Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
    /// X006      SY_MOCK
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchS02", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; //              JS sink platform T001c
        static bool TimeStep()
```

```
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P02.Boot
{
    /// &lt;summary&gt;
    ///       T001b       /  /       com.sy.core   ——
    /// SY_MOCK  =TimeService(mockMode)+SaveService(     )+LocaleService(     )
    /// Config        Ads  SDK  Mock——X006/X002a    platform T001c Channel
    /// Boot         Editor
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            // T001b core      SY_MOCK=    +0   T001c WX/TT
```

```
            ServiceLocator.Register&lt;ITimeService&gt;(new TimeService(null, null, mockMode: true, mockOffsetMs: 0));
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new SaveService());
            // U184 font-chain fix: wire a REAL fontSetLookup so MakeText helpers
            // resolve the bundled CJK subset (P02FontCJK) - device-safe Chinese
            // (LegacyRuntime.ttf has no CJK and WebGL has no OS font fallback).
            // X2315 slice D-2: BoldFont slot carries the serif title face
            // (P02TitleCJK, FT-002 Source Han Serif Bold subset) so title/engraving
            // slots render the art-spec row-7 "star-white serif" instead of sans.
            ServiceLocator.Register&lt;ILocaleService&gt;(new LocaleService(null, lang =&gt; new FontSet("P02FontCJK", "
            // T001c: full platform suite (mock in Editor/SY_MOCK; channel-gated WX/TT for real builds)
            // T006-B2: the shared AdCapPolicy is registered BEFORE the suite so the
            // rescue pre-check (HasCategoryBudget) and the suite's ShowRewarded
            // consumption read the same §  category counters.
            var adCap = new AdCapPolicy();
            ServiceLocator.Register(adCap);
            var suite = PlatformBootstrap.Create(adCap);
            ServiceLocator.Register&lt;IPlatformService&gt;(suite.Platform);
            // T015b-B: launch-entry query face (suites without a launch bridge
            // stay null -&gt; ShareReturn degrades to an organic open, zero events).
            if (suite.Launch != null) ServiceLocator.Register(suite.Launch);
            ServiceLocator.Register&lt;IAdsService&gt;(suite.Ads);
            ServiceLocator.Register&lt;IAdGrantService&gt;(suite.Grant);
            ServiceLocator.Register&lt;IAntiAddiction&gt;(suite.AntiAddiction);
            ServiceLocator.Register&lt;ILeaderboardService&gt;(suite.Leaderboard);
            ServiceLocator.Register&lt;IOpenDataBoard&gt;(suite.OpenData);
            ServiceLocator.Register&lt;IStorage&gt;(suite.Storage);
            ServiceLocator.Register&lt;IRecording&gt;(suite.Recording);
            ServiceLocator.Register&lt;ISubscribe&gt;(suite.Subscribe);
            ServiceLocator.Register&lt;ICloudService&gt;(suite.Cloud);
            // T001e: telemetry service with schema + dual-write sinks.
            // Editor/SY_MOCK: debug log sink; real channels add WxAnalytics + Tcb batch (X001a/X006).
            var telemetry = new Sy.Telemetry.TelemetryService();
            string cfgEventPath = System.IO.Path.Combine(
                System.IO.Path.GetFullPath(System.IO.Path.Combine(Application.dataPath, "..", "..", "..")),
                "Design", "configs", "P02", "cfg_event.json");
            if (System.IO.File.Exists(cfgEventPath))
                telemetry.LoadSchemaFromJson(System.IO.File.ReadAllText(cfgEventPath));
            telemetry.AddSink(new Sy.Telemetry.DebugLogSink());
            ServiceLocator.Register(telemetry);
            Logger.Info("BOOT", "services registered (T001e platform suite + telemetry, SY_MOCK)");
        }
    }
    public sealed class MockConfigService : IConfigService
    {
        public T GetTable&lt;T&gt;(string tableId) where T : class
            =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
        public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
        {
            table = null;
```

```
            return false;
        }
        public ConfigUpdateResult CheckUpdate(string manifestUrl)
            =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
        public void SwapPolicy(ConfigSwapPolicy policy) { }
        public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
    }
}
// MainUI.cs
using System.Collections.Generic;
using Sy.P02.Summon;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.P02.Newbie
{
    /// &lt;summary&gt;
    /// S02 main screen: Start Fate -&gt; S03_Battle (no ad points before M5), plus the
    /// permanent settings entry whose panel always shows the live summon odds tables
    /// (task T003 compliance: the disclosure must stay reachable at all times).
    /// UI is built programmatically at Awake (BattleView convention - no scene YAML drift).
    /// &lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        static readonly string[] TierLabels = { "White", "Blue", "Purple", "Gold" };
        static readonly Color[] TierColors =
        {
            new Color(0.91f, 0.89f, 0.85f),
            new Color(0.30f, 0.55f, 0.74f),
            new Color(0.62f, 0.36f, 0.76f),
            new Color(0.92f, 0.72f, 0.26f),
        };
        RectTransform _oddsPanel;
        RectTransform _defensePanel;
        InputField _defenseInput;
        Text _defenseHint;
        RectTransform _replayPanel;   // T014-B2b: shadow-replay watch entry
        InputField _replayInput;
        Text _replayHint;
        Text _purpleHint;   // T015-A4: purple_pick claim feedback under the button
        RectTransform _offlinePanel;   // T015-A5 UI half: offline_double return page
        Text _offlineHint;
        OfflineEarnings.OfflineClaim _offlineClaim;
        RectTransform _codexPanel;   // X921: S06 codex overlay (roster + lucky progress)
        Sprite _square;
        Sy.Telemetry.TelemetryService _telemetry;
        Sy.Platform.AdCapPolicy _adCap;   // T015-A4: direct 1/day pre-check (§  point 8)
        P02AudioHost _audioHost;   // X816 slice C: menu-face tap beats
        const string PurplePickPointId = "purple_pick";   // T015-A4: §  point 8 (direct purple grant)
        const string OfflineDoublePointId = "offline_double";   // T015-A5: §  point 6 (return-page x2)
```

```
const string HomeBannerPointId = "home_banner";   // X137 ad-mix: home banner lives on the main screen only
public void StartFate()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    SceneManager.LoadScene(NewbieSceneIds.Battle);
}
void Awake()
{
    Application.targetFrameRate = 30;
    ResolveTelemetry();
    // T015b-B K segment 2: read the launch query once per process; a
    // friend-card open fires evt_share(stage=card_opened) and leaves a
    // one-shot pending seed. Organic opens stay inactive (zero drift).
    ShareReturn.CaptureFromPlatform(_telemetry);
    // X137 ad-mix wiring: the home banner rides the main screen only
    // (a session without the ad service - direct test scene entry -
    // skips the call; the suite's no-ad-unit guard stays silent).
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var homeAds))
        homeAds.ShowBanner(HomeBannerPointId);
    if (!Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.AdCapPolicy&gt;(out _adCap))
        _adCap = new Sy.Platform.AdCapPolicy();   // degraded session: fallback cap (BattleLauncher convention)
    BuildSettingsEntry();
    BuildDestinyEntry();
    BuildDefenseEntry();
    BuildReplayEntry();
    BuildBoardEntry();
    BuildPurplePickEntry();
    BuildShareReturnEntry();
    BuildOfflineReturnEntry();
    BuildCodexEntry();
    ApplyMenuFramework();
    // X813 slice A: menu BGM (G11 paradigm) - null-safe, so a missing
    // catalog keeps direct scene entries silent and gates green.
    // X816 slice C volume pass: BGM sits one mix-tier under the SFX
    // layer so every menu tap reads through the music.
    _audioHost = P02AudioHost.Ensure("bgm_main", 0.55f);
}
/// &lt;summary&gt;W02 menu framework (X789): fullscreen generated backdrop behind
/// all menu UI, BattleView.BuildBackdrop convention. Skipped when the art
/// key is missing so the scene color-block fallback stays and headless
/// checks stay green (X027 no-crash fallback law).&lt;/summary&gt;
void ApplyMenuFramework()
{
    var art = Sy.Core.ArtRuntime.Get("P02_scene_menu_01");
    if (art == null) return;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(canvas.transform, false);
    var rt = (RectTransform)go.transform;
```

```
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        // The scene's solid Bg block sits at the bottom; the art rides right
        // above it so the fallback color stays visible when the art is absent.
        var bg = canvas.transform.Find("Bg");
        rt.SetSiblingIndex(bg != null ? bg.GetSiblingIndex() + 1 : 0);
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = art;
        image.color = Color.white;
        image.preserveAspect = true;
        image.raycastTarget = false;
    }
    /// &lt;summary&gt;Boot-registered telemetry when the boot scene ran first
    /// (BattleLauncher convention); the fallback keeps direct scene entry
    /// working for tests and editor play.&lt;/summary&gt;
    void ResolveTelemetry()
    {
        if (!Sy.Core.ServiceLocator.TryGet&lt;Sy.Telemetry.TelemetryService&gt;(out _telemetry))
        {
            _telemetry = new Sy.Telemetry.TelemetryService();
            _telemetry.AddSink(new Sy.Telemetry.DebugLogSink());
        }
    }
    /// &lt;summary&gt;T013-B: the Destiny Trial entry - visible only to the
    /// eligible gray cohort (original 5% server gray -&gt; local A/B,
    /// zero-server). The impression/click funnel rides evt_destiny; the
    /// button arms BattleLauncher's one-shot destiny flag so the battle
    /// opens on today's shared seed.&lt;/summary&gt;
    void BuildDestinyEntry()
    {
        if (!DestinyGate.IsEligible()) return;
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        if (canvas == null) return;
        var btnGo = new GameObject("DestinyButton", typeof(RectTransform), typeof(Image), typeof(Button));
        btnGo.transform.SetParent(canvas.transform, false);
        var btnRt = (RectTransform)btnGo.transform;
        btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
        btnRt.anchoredPosition = new Vector2(0, -400);
        btnRt.sizeDelta = new Vector2(460, 110);
        var img = btnGo.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = new Color(0.45f, 0.30f, 0.55f);
        MakeText(btnGo.transform, "Destiny Trial", 46, Vector2.zero, TextAnchor.MiddleCenter, 460, 110);
        btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(StartDestiny);
        _telemetry.Track(Sy.Telemetry.EventIds.Destiny, DestinyGate.EntryFields("entry_impression"));
    }
    void StartDestiny()
    {
        _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
```

```
    _telemetry.Track(Sy.Telemetry.EventIds.Destiny, DestinyGate.EntryFields("entry_click"));
    BattleLauncher.DestinyRunRequested = true;
    SceneManager.LoadScene(NewbieSceneIds.Battle);
}
// ——— T015b-B: share-card return entry (offline return flow, zero server) ———
/// &lt;summary&gt;Visible only when the app was opened from a friend's share
/// card with a valid seed. One tap arms BattleLauncher's one-shot
/// share-run flag and opens the battle on the friend's seed (K segment
/// 3); every other entry keeps its own seed and the pending seed stays
/// armed for a later rematch.&lt;/summary&gt;
void BuildShareReturnEntry()
{
    if (!ShareReturn.HasPendingSeed) return;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var btnGo = new GameObject("ShareReturnButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(0, -520);
    btnRt.sizeDelta = new Vector2(460, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.22f, 0.56f, 0.36f);
    MakeText(btnGo.transform, "Rematch Friend's Board", 40, Vector2.zero, TextAnchor.MiddleCenter, 460, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(StartShareReturn);
}
void StartShareReturn()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    BattleLauncher.PendingShareRunRequested = true;
    SceneManager.LoadScene(NewbieSceneIds.Battle);
}
// ——— T014-B: defense challenge entry (offline versus, zero server) ———
/// &lt;summary&gt;T014-B: the versus entry - visible to everyone (a core
/// share loop, not an A/B experiment). The panel imports a friend's
/// settle code or falls back to today's system preset; the funnel rides
/// evt_defense (entry_impression / entry_click / import_reject /
/// challenge_start).&lt;/summary&gt;
void BuildDefenseEntry()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var btnGo = new GameObject("DefenseButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(0, -560);
    btnRt.sizeDelta = new Vector2(460, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
```

```
    img.sprite = _square;
    img.color = new Color(0.22f, 0.45f, 0.52f);
    MakeText(btnGo.transform, "Defense Challenge", 44, Vector2.zero, TextAnchor.MiddleCenter, 460, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(ToggleDefensePanel);
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, DefenseFields("entry_impression", 0, "menu"));
    BuildDefensePanel(canvas.transform);
}
void ToggleDefensePanel()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    _defensePanel.gameObject.SetActive(!_defensePanel.gameObject.activeSelf);
}
void BuildDefensePanel(Transform canvasRoot)
{
    _defensePanel = MakeOverlayPanel(canvasRoot, "DefensePanel", new Color(0.02f, 0.05f, 0.09f, 0.95f));
    MakeText(_defensePanel, "Defense Challenge", 66, new Vector2(0, 740), TextAnchor.MiddleCenter, 1040, 100, ti
    MakeText(_defensePanel, "Import a friend's settle code - same seed, same battle, beat their bar",
        40, new Vector2(0, 600), TextAnchor.MiddleCenter, 1040, 90).color = new Color(0.62f, 0.70f, 0.78f);
    _defenseInput = MakeCodeInput(_defensePanel, new Vector2(0, 440));
    _defenseHint = MakeText(_defensePanel, "", 36, new Vector2(0, 300), TextAnchor.MiddleCenter, 1040, 70);
    _defenseHint.color = new Color(0.92f, 0.55f, 0.45f);
    MakeActionButton(_defensePanel, "Challenge!", new Vector2(0, 160), OnChallengeClicked);
    MakeActionButton(_defensePanel, "Today's Defense", new Vector2(0, 0), OnSystemDefenseClicked);
    MakeActionButton(_defensePanel, "Close", new Vector2(0, -160), () =&gt; _defensePanel.gameObject.SetActive(f
    _defensePanel.gameObject.SetActive(false);
}
/// &lt;summary&gt;Friend-code path: decode through the same import validation
/// the gate drives (checksum + table-release pin), then arm the battle
/// scene's one-shot challenge snapshot.&lt;/summary&gt;
void OnChallengeClicked()
{
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, DefenseFields("entry_click", 0, "menu"));
    string code = _defenseInput != null ? _defenseInput.text : "";
    var snap = DefenseChallenge.ImportCode(code, out string reject);
    if (snap == null)
    {
        _defenseHint.text = reject == "version"
            ? "This code is from another game version"
            : "Invalid code";
        _telemetry.Track(Sy.Telemetry.EventIds.Defense, DefenseFields("import_reject", 0, "friend"));
        return;
    }
    StartDefenseChallenge(snap, "friend");
}
/// &lt;summary&gt;System-preset fallback (task-book "empty friend pool"):
/// today's preset rides the exact same snapshot/replay path a friend
/// code takes - no special-casing downstream.&lt;/summary&gt;
void OnSystemDefenseClicked()
{
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, DefenseFields("entry_click", 0, "menu"));
```

```
    StartDefenseChallenge(SystemDefenseTable.ToSnapshot(SystemDefenseTable.ForToday()), "system");
}
void StartDefenseChallenge(DefenseSnapshot snap, string source)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, DefenseFields("challenge_start", snap.Seed, source));
    BattleLauncher.PendingDefenseChallenge = snap;
    BattleLauncher.PendingDefenseSource = source;
    SceneManager.LoadScene(NewbieSceneIds.Battle);
}
static Dictionary&lt;string, string&gt; DefenseFields(string action, int seed, string source)
{
    return new Dictionary&lt;string, string&gt;
    {
        { "action", action },
        { "seed", seed.ToString() },
        { "source", source },
    };
}
// ——— T014-B2b: shadow-replay watch entry (the "see how that code
//      played out" half of the share loop, zero server) ———
/// &lt;summary&gt;T014-B2b: the watch entry - imports a settle code and
/// hands it to the battle scene as a pending replay; the funnel rides
/// evt_defense replay actions (impression/click/reject/start).&lt;/summary&gt;
void BuildReplayEntry()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var btnGo = new GameObject("ReplayButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(0, -720);
    btnRt.sizeDelta = new Vector2(460, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.50f, 0.36f, 0.16f);
    MakeText(btnGo.transform, "Shadow Replay", 44, Vector2.zero, TextAnchor.MiddleCenter, 460, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(ToggleReplayPanel);
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, ReplayFields("replay_impression", 0, "menu"));
    BuildReplayPanel(canvas.transform);
}
void ToggleReplayPanel()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    _replayPanel.gameObject.SetActive(!_replayPanel.gameObject.activeSelf);
}
void BuildReplayPanel(Transform canvasRoot)
{
    _replayPanel = MakeOverlayPanel(canvasRoot, "ReplayPanel", new Color(0.02f, 0.05f, 0.09f, 0.95f));
    MakeText(_replayPanel, "Shadow Replay", 66, new Vector2(0, 740), TextAnchor.MiddleCenter, 1040, 100, title:
```

```
    MakeText(_replayPanel, "Import a settle code - watch the shadow re-fight that battlefield",
        40, new Vector2(0, 600), TextAnchor.MiddleCenter, 1040, 90).color = new Color(0.62f, 0.70f, 0.78f);
    _replayInput = MakeCodeInput(_replayPanel, new Vector2(0, 440));
    _replayHint = MakeText(_replayPanel, "", 36, new Vector2(0, 300), TextAnchor.MiddleCenter, 1040, 70);
    _replayHint.color = new Color(0.92f, 0.55f, 0.45f);
    MakeActionButton(_replayPanel, "Watch!", new Vector2(0, 160), OnWatchClicked);
    MakeActionButton(_replayPanel, "Close", new Vector2(0, -20), () =&gt; _replayPanel.gameObject.SetActive(fals
    _replayPanel.gameObject.SetActive(false);
}
/// &lt;summary&gt;Same import validation as the challenge panel (checksum +
/// table-release pin); the accepted snapshot arms the battle scene's
/// one-shot replay pair instead of the challenge pair.&lt;/summary&gt;
void OnWatchClicked()
{
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, ReplayFields("replay_click", 0, "menu"));
    string code = _replayInput != null ? _replayInput.text : "";
    var snap = DefenseChallenge.ImportCode(code, out string reject);
    if (snap == null)
    {
        _replayHint.text = reject == "version"
            ? "This code is from another game version"
            : "Invalid code";
        _telemetry.Track(Sy.Telemetry.EventIds.Defense, ReplayFields("replay_reject", 0, "friend"));
        return;
    }
    _telemetry.Track(Sy.Telemetry.EventIds.Defense, ReplayFields("replay_start", snap.Seed, "friend"));
    DefenseReplay.SetPending(snap, "friend");
    SceneManager.LoadScene(NewbieSceneIds.Battle);
}
static Dictionary&lt;string, string&gt; ReplayFields(string action, int seed, string source)
{
    return new Dictionary&lt;string, string&gt;
    {
        { "action", action },
        { "seed", seed.ToString() },
        { "source", source },
    };
}
// ——— T013-B2b: challenge-board entry (the destiny feature's friend
//      face - same 5% cohort gate as the destiny entry) ———
/// &lt;summary&gt;T013-B2b: the board entry - visible to the destiny cohort
/// only (the board is the destiny challenge's share face; holdouts
/// cannot post a day-best, so the page would tease content they
/// cannot reach). Opens S07, where the WeChat open-data layer paints
/// friends' day-best; the funnel rides evt_destiny board actions.&lt;/summary&gt;
void BuildBoardEntry()
{
    if (!DestinyGate.IsEligible()) return;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
```

```
    var btnGo = new GameObject("BoardButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(0, -880);
    btnRt.sizeDelta = new Vector2(460, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.85f, 0.62f, 0.22f);
    MakeText(btnGo.transform, "Challenge Board", 44, Vector2.zero, TextAnchor.MiddleCenter, 460, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(StartBoard);
    _telemetry.Track(Sy.Telemetry.EventIds.Destiny, BoardUI.BoardFields("board_impression"));
}
void StartBoard()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    _telemetry.Track(Sy.Telemetry.EventIds.Destiny, BoardUI.BoardFields("board_click"));
    SceneManager.LoadScene(NewbieSceneIds.Board);
}
// ——— T015-A4: purple_pick entry (§  point 8, direct category) ———
/// &lt;summary&gt;The persistent main-screen purple entry (   ): a rewarded
/// ad grants one random purple unit that joins the next standard Fate
/// run's board. Everyone sees it (a core §  revenue point, not an A/B
/// experiment); the direct category caps it at 1/day, and the claim
/// hint stays honest instead of a dead ad call once claimed.&lt;/summary&gt;
void BuildPurplePickEntry()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var btnGo = new GameObject("PurplePickButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(-400, 850);
    btnRt.sizeDelta = new Vector2(300, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.62f, 0.36f, 0.76f);   // the tier-purple the ad grants
    MakeText(btnGo.transform, "Free Purple", 40, Vector2.zero, TextAnchor.MiddleCenter, 300, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(OnPurplePickClicked);
    _purpleHint = MakeText(canvas.transform, "", 28, new Vector2(-400, 720), TextAnchor.MiddleCenter, 560, 90);
    _purpleHint.color = new Color(0.92f, 0.83f, 0.55f);
    TrackPurplePick("offer");
}
async void OnPurplePickClicked()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    // §  pre-offer gate: the direct category is 1/day - once claimed
    // the honest answer is the hint, not a dead ad call.
    string day = Sy.Platform.AdCapPolicy.TodayKey();
```

```
    if (!_adCap.HasCategoryBudget(day, Sy.Platform.AdCategories.Direct))
    {
        _purpleHint.text = "Today's purple is already claimed - come back tomorrow";
        return;
    }
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(PurplePickPointId);
        if (!result.Completed)
        {
            _purpleHint.text = "Ad did not finish - the purple is still waiting";
            TrackPurplePick("ad_fail");
            return;
        }
    }
    else
    {
        // Degraded session (no boot): grant locally against the fallback cap.
        _adCap.TryConsumeRewarded(day, Sy.Platform.AdCategories.CategoryOfPoint(PurplePickPointId));
    }
    BattleLauncher.PendingPurplePick = true;
    _purpleHint.text = "A purple unit joins your next Fate run!";
    TrackPurplePick("granted");
}
void TrackPurplePick(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
        { "state", state },
        { "pointId", PurplePickPointId },
        { "reason", "direct_purple" },
    });
}
// ——— T015-A5 UI half: offline_double return page (§  point 6, double category) ———
/// &lt;summary&gt;The return-visit claim page (offline_double): shows once
/// when the shop banked at least one full hour while the player was
/// away. Collect banks the frozen pending coins through the real
/// wallet; the rewarded ad doubles them (double category, 5/day
/// shared with settle_double/summon_reroll). A visit with nothing
/// claimable re-arms the seen stamp so sub-hour stops never tease a
/// dead page; an ignored claim keeps accruing from the same stamp
/// (unclaimed time is never punished).&lt;/summary&gt;
void BuildOfflineReturnEntry()
{
    long nowMs = System.DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    var claim = OfflineEarnings.OfflineClaim.Open(nowMs);
    if (!claim.Claimable)
    {
        OfflineEarnings.Stamp(nowMs);
        return;
```

```
    }
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    _offlineClaim = claim;
    _offlinePanel = MakeOverlayPanel(canvas.transform, "OfflineReturnPanel", new Color(0.02f, 0.05f, 0.09f, 0.95
    MakeText(_offlinePanel, "Welcome back!", 66, new Vector2(0, 740), TextAnchor.MiddleCenter, 1040, 100, title:
    MakeText(_offlinePanel, "While you were away the shop kept the incense warm",
        40, new Vector2(0, 600), TextAnchor.MiddleCenter, 1040, 90).color = new Color(0.62f, 0.70f, 0.78f);
    MakeText(_offlinePanel, "+" + claim.Pending + " lucky coins", 56, new Vector2(0, 470), TextAnchor.MiddleCent
    _offlineHint = MakeText(_offlinePanel, "", 34, new Vector2(0, 330), TextAnchor.MiddleCenter, 1040, 80);
    _offlineHint.color = new Color(0.92f, 0.83f, 0.55f);
    MakeActionButton(_offlinePanel, "Watch Ad: x2", new Vector2(0, 170), OnOfflineDoubleClicked);
    MakeActionButton(_offlinePanel, "Collect", new Vector2(0, 10), OnOfflineCollectClicked);
    MakeActionButton(_offlinePanel, "Close", new Vector2(0, -150), () =&gt; _offlinePanel.gameObject.SetActive(f
    TrackOffline("offer");
}
/// &lt;summary&gt;Plain collect: banks the frozen pending coins as-is.&lt;/summary&gt;
void OnOfflineCollectClicked()
{
    int granted = _offlineClaim != null ? _offlineClaim.TryCollect(false) : -1;
    if (granted &lt; 0) return;
    _offlineHint.text = "+" + granted + " coins banked - the shop thanks you";
    TrackOffline("granted");
}
/// &lt;summary&gt;Rewarded-ad path: the double category is 5/day shared, so
/// the pre-offer gate keeps the honest hint instead of a dead ad
/// call once the day's x2 budget is gone (purple_pick convention);
/// a session without the ad service consumes locally against the
/// fallback cap.&lt;/summary&gt;
async void OnOfflineDoubleClicked()
{
    if (_offlineClaim == null || _offlineClaim.Consumed)
    {
        _offlineHint.text = "Already collected - the coins are in the wallet";
        return;
    }
    TrackOffline("accept");
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (!_adCap.HasCategoryBudget(day, Sy.Platform.AdCategories.Double))
    {
        _offlineHint.text = "Today's x2 bonus is used up - collect as is";
        TrackOffline("blocked");
        return;
    }
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(OfflineDoublePointId);
        if (!result.Completed)
        {
            _offlineHint.text = "Ad did not finish - the coins still wait";
```

```
            TrackOffline("ad_fail");
            return;
        }
    }
    else
    {
        // Degraded session (no boot): grant locally against the fallback cap.
        _adCap.TryConsumeRewarded(day, Sy.Platform.AdCategories.CategoryOfPoint(OfflineDoublePointId));
    }
    int granted = _offlineClaim.TryCollect(true);
    if (granted &lt; 0) return;
    _offlineHint.text = "x2! +" + granted + " coins banked";
    TrackOffline("granted");
}
void TrackOffline(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
        { "state", state },
        { "pointId", OfflineDoublePointId },
        { "reason", "offline_return" },
    });
}
// ——— X921: S06 codex entry (roster + lucky progress, zero server) ———
/// &lt;summary&gt;The codex entry - everyone sees it (the collection book
/// is a core anticipation loop, not an A/B experiment). Opens the S06
/// overlay: live unit roster with tier colors plus the lucky wallet
/// progress the settle redemption writes into.&lt;/summary&gt;
void BuildCodexEntry()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var btnGo = new GameObject("CodexButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(400, 700);
    btnRt.sizeDelta = new Vector2(300, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.16f, 0.45f, 0.40f);
    MakeText(btnGo.transform, "Codex", 44, Vector2.zero, TextAnchor.MiddleCenter, 300, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(ToggleCodexPanel);
    _codexPanel = CodexView.Build(canvas.transform);
    MakeActionButton(_codexPanel, "Close", new Vector2(0, -680), () =&gt; _codexPanel.gameObject.SetActive(false
}
void ToggleCodexPanel()
{
    _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
    _codexPanel.gameObject.SetActive(!_codexPanel.gameObject.activeSelf);
```

```
}
InputField MakeCodeInput(RectTransform parent, Vector2 pos)
{
    var go = new GameObject("CodeInput", typeof(RectTransform), typeof(Image), typeof(InputField));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(960, 110);
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.12f, 0.16f, 0.22f);
    var input = go.GetComponent&lt;InputField&gt;();
    var text = MakeText(go.transform, "", 34, Vector2.zero, TextAnchor.MiddleLeft, 940, 100);
    text.supportRichText = false;
    input.textComponent = text;
    var placeholder = MakeText(go.transform, "FG1-.....", 34, Vector2.zero, TextAnchor.MiddleLeft, 940, 100);
    placeholder.color = new Color(0.45f, 0.50f, 0.58f);
    input.placeholder = placeholder;
    return input;
}
void BuildSettingsEntry()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    _square = MakeSquareSprite();
    var btnGo = new GameObject("OddsButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(canvas.transform, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(400, 850);
    btnRt.sizeDelta = new Vector2(300, 110);
    var img = btnGo.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.18f, 0.30f, 0.42f);
    MakeText(btnGo.transform, "Odds", 48, Vector2.zero, TextAnchor.MiddleCenter, 300, 110);
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(ToggleOddsPanel);
    BuildOddsPanel(canvas.transform);
}
/// &lt;summary&gt;Permanent odds disclosure table (        ): both weight
/// tables are generated from the same config the battle pool rolls, so the
/// published values can only equal the real pool values.&lt;/summary&gt;
void BuildOddsPanel(Transform canvasRoot)
{
    var cfg = Sy.P02.Summon.SummonGreyGate.LiveConfig();   // T023-B: disclosure follows the grey-resolved relea
    _oddsPanel = MakeOverlayPanel(canvasRoot, "OddsPanel", new Color(0.02f, 0.05f, 0.09f, 0.95f));
    // W08 settings art (X920 consume / X921 wiring): fullscreen
    // generated backdrop as the first child of the settings panel -
    // above the flat dark fallback, under the disclosure tables
    // (BuildSettlePanel convention). Skipped when the art key is
```

```
        // missing so the color-block fallback stays and headless checks
        // stay green (X027 no-crash fallback law).
        var settingsArt = Sy.Core.ArtRuntime.Get("P02_scene_settings_01");
        if (settingsArt != null)
        {
            var artGo = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
            artGo.transform.SetParent(_oddsPanel, false);
            var artRt = (RectTransform)artGo.transform;
            artRt.anchorMin = Vector2.zero;
            artRt.anchorMax = Vector2.one;
            artRt.offsetMin = artRt.offsetMax = Vector2.zero;
            artRt.SetAsFirstSibling();
            var artImage = artGo.GetComponent&lt;Image&gt;();
            artImage.sprite = settingsArt;
            artImage.color = Color.white;
            artImage.preserveAspect = true;
            artImage.raycastTarget = false;
        }
        MakeText(_oddsPanel, "Summon Odds Disclosure", 66, new Vector2(0, 740), TextAnchor.MiddleCenter, 1040, 100,
        MakeText(_oddsPanel, "Summon pool", 50, new Vector2(0, 580), TextAnchor.MiddleCenter, 900, 80)
            .color = new Color(0.62f, 0.70f, 0.78f);
        for (int i = 0; i &lt; TierLabels.Length; i++)
        {
            var row = MakeText(_oddsPanel, TierLabels[i] + "  " + cfg.MainPercent((Tier)i), 54,
                new Vector2(0, 480 - i * 110), TextAnchor.MiddleCenter, 900, 96);
            row.color = TierColors[i];
        }
        MakeText(_oddsPanel, "Ad pool (ad summons never roll White)", 44, new Vector2(0, -20), TextAnchor.MiddleCent
            .color = new Color(0.62f, 0.70f, 0.78f);
        for (int i = 0; i &lt; TierLabels.Length; i++)
        {
            var row = MakeText(_oddsPanel, TierLabels[i] + "  " + cfg.AdPercent((Tier)i), 54,
                new Vector2(0, -120 - i * 110), TextAnchor.MiddleCenter, 900, 96);
            row.color = TierColors[i];
        }
        var pity = MakeText(_oddsPanel, cfg.PityRuleText(), 42, new Vector2(0, -620), TextAnchor.MiddleCenter, 1040,
        pity.color = new Color(0.92f, 0.83f, 0.55f);
        // T024 compliance: explicit AIGC disclosure rides the permanent
        // settings table (platform review requires a visible AI label).
        var aigc = MakeText(_oddsPanel, LocaleService.Get("aigc_disclosure"), 36,
            new Vector2(0, -707), TextAnchor.MiddleCenter, 1040, 60);
        aigc.color = new Color(0.62f, 0.70f, 0.78f);
        MakeActionButton(_oddsPanel, "Close", new Vector2(0, -800), () =&gt; _oddsPanel.gameObject.SetActive(false))
        _oddsPanel.gameObject.SetActive(false);
    }
    void ToggleOddsPanel()
    {
        _audioHost?.PlaySfx("sfx_ui_click", 0.8f);   // X816 slice C: menu-face beat
        _oddsPanel.gameObject.SetActive(!_oddsPanel.gameObject.activeSelf);
    }
```

```
// ——— UI helpers (BattleView conventions, duplicated to stay scene-agnostic) ———
RectTransform MakeOverlayPanel(Transform parent, string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    return rt;
}
void MakeActionButton(RectTransform parent, string text, Vector2 pos, System.Action onClick)
{
    var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(420, 120);
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = new Color(0.29f, 0.55f, 0.55f);
    MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
    // X816 slice C: every panel action button carries the tap beat in
    // one place - the named handlers stay single-responsibility.
    go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt;
    {
        _audioHost?.PlaySfx("sfx_ui_click", 0.8f);
        onClick();
    });
}
static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h,
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent&lt;Text&gt;();
    t.font = title ? UiFont.ResolveTitle() : UiFont.Resolve(); // U184 CJK chain; X2315 D-2: title face = serif
    t.raycastTarget = false;                // U184: 17 doc 1.2 - labels never eat taps
    t.text = text;
    t.fontSize = size;
    t.alignment = anchor;
    t.color = new Color(0.91f, 0.89f, 0.85f);
    t.supportRichText = true;
```

```
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// AssemblyInfo.cs
using System.Runtime.CompilerServices;
// T005: editor gates (Sy.P02.Editor) construct exhaustive board states through
// BattleSim.InjectUnitForTests; gameplay assemblies never see the internals.
// X1321 (12th doc sec 12 batch A): the PlayMode input-chain domain needs the
// same state-injection seam to stage the recycle gate deterministically.
[assembly: InternalsVisibleTo("Sy.P02.Editor")]
[assembly: InternalsVisibleTo("Sy.P02.PlayModeTests")]
// LaunchComplianceFace.cs
using Sy.P02.Newbie;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace Sy.P02.Boot
{
    /// &lt;summary&gt;
    /// U185 launch compliance face (19-doc section-2 group F): the S00_Boot
    /// screen doubles as the launch page - the full Health Game Advice text
    /// plus the approval info block (copyright holder, software-copyright
    /// registration, filing slot, age rating). Every player-facing string
    /// rides cfg_i18n (launch.* / approval.*), so the submit-time filing
    /// number is a table edit, never a code change. Non-blocking by design:
    /// a minimum display window with tap-to-enter keeps the advice visible
    /// on every real launch while scene flow stays driven (flow checks run
    /// sim-level and never pass through this scene - zero drift).
    /// &lt;/summary&gt;
    public sealed class LaunchComplianceFace
    {
        public const float MinDisplaySeconds = 3f;
        public bool EnterTapped { get; private set; }
        /// &lt;summary&gt;Builds the launch page onto its own overlay canvas (the
        /// boot scene ships camera + runner only). Safe to call once.&lt;/summary&gt;
        public static LaunchComplianceFace Build()
        {
            var face = new LaunchComplianceFace();
            var canvasGo = new GameObject("LaunchCanvas", typeof(Canvas), typeof(CanvasScaler), typeof(GraphicRaycaster)
            var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
```

```
canvas.renderMode = RenderMode.ScreenSpaceOverlay;
var scaler = canvasGo.GetComponent&lt;CanvasScaler&gt;();
scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;   // U184: 17-doc canonical scaler
scaler.referenceResolution = new Vector2(1080f, 1920f);
scaler.screenMatchMode = CanvasScaler.ScreenMatchMode.MatchWidthOrHeight;
scaler.matchWidthOrHeight = 0f;
if (UnityEngine.Object.FindObjectOfType&lt;EventSystem&gt;() == null)
{
    var es = new GameObject("LaunchEventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
    es.transform.SetParent(canvasGo.transform, false);
}
var root = MakePanel(canvas.transform, "LaunchCompliancePage", new Color(0.02f, 0.05f, 0.09f, 1f));
var title = MakeText(root, LocaleService.Get("launch.title"), 76,
    new Vector2(0, 800), TextAnchor.MiddleCenter, 1040, 110);
title.color = new Color(0.91f, 0.89f, 0.85f);
var adviceLabel = MakeText(root, LocaleService.Get("launch.health_advice_label"), 44,
    new Vector2(0, 620), TextAnchor.MiddleCenter, 1040, 80);
adviceLabel.color = new Color(0.92f, 0.83f, 0.55f);
var advice = MakeText(root, LocaleService.Get("launch.health_advice"), 40,
    new Vector2(0, 430), TextAnchor.MiddleCenter, 1000, 280);
advice.color = new Color(0.91f, 0.89f, 0.85f);
var approvalLabel = MakeText(root, LocaleService.Get("approval.title"), 40,
    new Vector2(0, 140), TextAnchor.MiddleCenter, 1040, 70);
approvalLabel.color = new Color(0.92f, 0.83f, 0.55f);
string[] rows =
{
    LocaleService.Get("approval.game_name"),
    LocaleService.Get("approval.copyright_holder"),
    LocaleService.Get("approval.software_copyright"),
    LocaleService.Get("approval.filing_no"),
    LocaleService.Get("approval.age_rating"),
    LocaleService.Get("approval.developer"),
};
for (int i = 0; i &lt; rows.Length; i++)
{
    var row = MakeText(root, rows[i], 34,
        new Vector2(0, 20 - i * 120), TextAnchor.MiddleCenter, 1040, 100);
    row.color = new Color(0.62f, 0.70f, 0.78f);
}
var btnGo = new GameObject("EnterButton", typeof(RectTransform), typeof(Image), typeof(Button));
btnGo.transform.SetParent(root, false);
var btnRt = (RectTransform)btnGo.transform;
btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
btnRt.anchoredPosition = new Vector2(0, -760);
btnRt.sizeDelta = new Vector2(460, 120);
var img = btnGo.GetComponent&lt;Image&gt;();
img.sprite = MakeSquareSprite();
img.color = new Color(0.29f, 0.55f, 0.55f);
MakeText(btnGo.transform, LocaleService.Get("launch.enter"), 46,
    Vector2.zero, TextAnchor.MiddleCenter, 460, 120);
```

```
            btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; face.EnterTapped = true);
            return face;
        }
        static RectTransform MakePanel(Transform parent, string name, Color color)
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = Vector2.zero;
            rt.anchorMax = Vector2.one;
            rt.offsetMin = rt.offsetMax = Vector2.zero;
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = MakeSquareSprite();
            image.color = color;
            return rt;
        }
        static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
        {
            var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(w, h);
            var t = go.GetComponent&lt;Text&gt;();
            t.font = UiFont.Resolve();               // U184: FontSet-driven CJK chain
            t.raycastTarget = false;                 // 17-doc 1.2: labels never eat taps
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// UiFont.cs
using System;
using Sy.Core;
using UnityEngine;
namespace Sy.P02
{
```

```
/// &lt;summary&gt;
/// U184 font-chain fix (audit B1/B2): resolve the UI font through the
/// locale FontSet instead of hardcoding the engine built-in font.
/// Why: LegacyRuntime.ttf carries no CJK glyphs and WebGL/WeChat builds
/// have no OS font fallback, so every Chinese label would render blank
/// on device (editor stays green via OS fallback - a false-comfort
/// mask). The bundled subset asset P02FontCJK.ttf (Noto Sans SC OFL,
/// subset to this game's own character census, 168KB) covers the whole
/// UI vocabulary; X027 law: missing asset falls back to the built-in
/// font + one warning, never breaks a build or gate.
/// &lt;/summary&gt;
public static class UiFont
{
    const string Fallback = "LegacyRuntime.ttf";
    static bool _warned;
    public static Font Resolve()
    {
        string name = null;
        try
        {
            if (ServiceLocator.TryGet&lt;ILocaleService&gt;(out var locale))
                name = locale.GetFontSet().NormalFont;
        }
        catch (Exception) { name = null; }
        if (!string.IsNullOrEmpty(name) &amp;&amp; name != Fallback)
        {
            var font = Resources.Load&lt;Font&gt;("Fonts/" + name);
            if (font != null) return font;
            if (!_warned)
            {
                Sy.Core.Logger.Warn("UIFONT", "font asset missing, fallback LegacyRuntime: " + name);
                _warned = true;
            }
        }
        return Resources.GetBuiltinResource&lt;Font&gt;(Fallback);
    }
    /// &lt;summary&gt;
    /// X2315 slice D-2: title/engraving face (art spec row 7 star-white serif).
    /// Reads the BoldFont slot; any failure falls back to Resolve() so a
    /// missing serif asset degrades to the CJK-safe normal font, never to
    /// LegacyRuntime's tofu chain (X027 law).
    /// &lt;/summary&gt;
    public static Font ResolveTitle()
    {
        string name = null;
        try
        {
            if (ServiceLocator.TryGet&lt;ILocaleService&gt;(out var locale))
                name = locale.GetFontSet().BoldFont;
        }
```

```
            catch (Exception) { name = null; }
            if (!string.IsNullOrEmpty(name) &amp;amp;&amp;amp; name != Fallback)
            {
                var font = Resources.Load&amp;lt;Font&amp;gt;("Fonts/" + name);
                if (font != null) return font;
                Sy.Core.Logger.Warn("UIFONT", "title font missing, fallback normal: " + name);
            }
            return Resolve();
        }
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
// X137: keeps the return-subscribe call site in the natural short form
// (Request(SubscribeType...)) - DouyinFaceCheck B4 greps that form.
using SubscribeType = Sy.Platform.SubscribeType;
namespace Sy.P02.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S02_Main";
        public const string Battle = "S03_Battle";
        public const string Board = "S07_Board";   // T013-B2b: destiny friend board
    }
    /// &amp;lt;summary&amp;gt;
    /// S03_Battle       BattleSim+TutorialFlow+BattleView   Tick
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
    /// HUD                     6
    /// &amp;lt;/summary&amp;gt;
    public sealed class BattleLauncher : MonoBehaviour
    {
        BattleSim _sim;
        TutorialFlow _tutorial;
        BattleView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        Sy.Platform.AdCapPolicy _adCap;   // T006-B2: rescue budget pre-check (§  4/day shared)
        Sy.Platform.IOpenDataBoard _openData;   // T013-B2: friend-board KV face (null without boot = submit no-op)
        LuckyWallet _wallet;              // T012-B: cross-run lucky-coin wallet (local authority)
        int _lastWave = -1, _lastLife = -1, _lastCrystals = -1, _lastKills = -1;
        bool _oddsAcknowledged;   // task T003: disclosure precedes the session's first summon
        int _starCardAb = -1;      // T009-B X009 local A/B bucket: 0 = A(2 cards), 1 = B(3 cards)
        P02AudioHost _audioHost;  // X813 slice A: scene-local audio host (battle BGM + event SFX)
        const string RescuePointId = "clear_wave";   // T006-B2: §  point 2
        const string ShieldPointId = "shield";       // T015-A1: §  point 1 (lethal guard)
        const string SettleDoublePointId = "settle_double";   // T015-A2: §  point 4 (settle coins x2)
        const string RerollPointId = "summon_reroll";         // T015-A2: §  point 5 (ad-pool bonus draw)
        const string ExpandPointId = "board_expand";          // T015-A3: §  point 3 (board +1 capacity)
        int _settleBonus;   // T015-A2: the run's banked coins the accepted double re-deposits
        Sy.Social.ShareCard _shareSpec;      // T015b: the composed card's source spec
```

```
Sy.Social.ComposeResult _shareCard;  // T015b: the composed brag card (png path/bytes)
Sy.Platform.IRecording _recording;   // P02-G4: highlight-video share face (M3 gate dual entry)
const string AbBucketKey = "p02_ab_star_card";
static readonly int[] TutorialOfferRows = { 0, 1 };   // teaching pause keeps the fixed pair
// T013-A: the day's destiny run is requested by the menu entry (T013-B
// wires the button behind the local A/B bucket). One-shot static so the
// battle scene can be opened directly in tests; default false keeps
// every existing entry path byte-identical.
public static bool DestinyRunRequested;
// T014-B: a versus challenge armed by the menu import panel (friend
// code or today's system preset). One-shot statics consumed in Awake;
// both null keeps every existing entry path byte-identical. A challenge
// takes seed precedence over destiny - the panel is the freshest
// explicit user action.
public static DefenseSnapshot PendingDefenseChallenge;
public static string PendingDefenseSource;
DefenseSnapshot _defense;    // T014-B: the imported bar this run challenges
string _defenseSource;      // T014-B: "friend" | "system" for evt_defense
// T015-A4: the main screen's purple_pick rewarded ad (§  point 8,
// direct 1/day) arms one purple unit for the next standard run.
// Shared-seed runs never take the grant (destiny = the same battle for
// everyone; versus/watch runs must recompute byte-identical), so on
// those entries the arm simply stays pending for the next standard
// run - the player's ad is never wasted. Default false keeps every
// existing entry path byte-identical.
public static bool PendingPurplePick;
// T015b-B: the main screen's share-return entry (friend-card rematch)
// armed this run with the shared seed. One-shot, consumed in Awake;
// default false keeps every existing entry path byte-identical. The
// share-return run is the recipient's own fresh battle on the friend's
// seed (not a byte-identical recompute), so grants stay fair game.
public static bool PendingShareRunRequested;
// T014-B2b: replay watch mode - the shadow re-fights an imported
// code's battlefield at the policy's own fixed cadence, so the watched
// settle IS the recomputed verdict (byte-identical to Replay()).
DefenseSnapshot _replay;
string _replaySource;       // "friend" for the evt_defense replay funnel
bool _watching;
float _watchAccum;
float _watchTime;
int _watchGuard;
const float WatchStepSeconds = 0.05f;   // DefenseReplay.ShadowPolicy step
const float WatchMaxSeconds = 300f;      // same settle window as the policy
void Awake()
{
    Application.targetFrameRate = 30;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    // Telemetry resolves before the seed chain (T015b-B): the shared-seed
    // take fires evt_share(stage=return_started) right where the seed is
    // picked. Boot-registered instance when the boot scene ran first,
```

```
// fallback keeps direct scene entry working (MainUI convention).
if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Telemetry.TelemetryService&gt;(out var bootTelemetry))
    _telemetry = bootTelemetry;
else
{
    _telemetry = new Sy.Telemetry.TelemetryService();
    _telemetry.AddSink(new TelemetryDebugSink());
}
// P02-G4: the platform recording face (boot-registered; direct
// battle-scene entry degrades to null = no clip offers, same
// no-dead-button law as the brag card).
_recording = Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IRecording&gt;(out var bootRecording) ? bootRecord
// T009-B X009 local A/B (X032 zero-server ruling): a one-time persistent
// device bucket fixes the formal offer size - A = 2 (table baseline),
// B = 3. The pickCount rides evt_star_card so the comparison is offline
// data, no server involvement.
_starCardAb = PlayerPrefs.GetInt(AbBucketKey, -1);
if (_starCardAb &lt; 0)
{
    _starCardAb = UnityEngine.Random.Range(0, 2);
    PlayerPrefs.SetInt(AbBucketKey, _starCardAb);
}
// T023-B: fix the grey cohort bucket once per install, before the
// seed chain and next to the star-card roll (same one-time
// PlayerPrefs pick). While the channel is disarmed the call is a
// no-op for gameplay; an armed reviewed build resolves the
// treatment cohort from this stable id.
Sy.P02.Summon.SummonGreyGate.Bucket();
int offerSize = _starCardAb == 0 ? StarCardTable.PickCountDefault : 3;
// X031 random level: a fresh random seed on every scene entry, so each
// playtest session verifies a different level; the seed is shown on the
// HUD, and the same seed always reproduces the same level.
// T013-A destiny run: the seed comes from the calendar date instead
// (same day = same battle on every device, zero server); the flag is
// consumed once so a later scene entry returns to the random mode.
bool destiny = DestinyRunRequested;
DestinyRunRequested = false;
_defense = PendingDefenseChallenge;   // T014-B: one-shot consume
_defenseSource = PendingDefenseSource;
PendingDefenseChallenge = null;
PendingDefenseSource = null;
// T014-B2b: one-shot replay handoff (same contract as the challenge
// pair); only one can ever be armed per menu action, and an armed
// challenge keeps seed precedence.
_watching = DefenseReplay.ConsumePending(out _replay, out _replaySource);
int seed = _defense != null ? _defense.Seed
    : _replay != null ? _replay.Seed
    : destiny ? DestinyPin.SeedForToday()
    : UnityEngine.Random.Range(int.MinValue, int.MaxValue);
// T015b-B K segment 3: the share-return entry (main-screen rematch
```

```
            // button) re-fights the friend card's seed exactly once; explicit
            // imports keep precedence and every other path returns the seed
            // above unchanged.
            bool shareRun = PendingShareRunRequested;
            PendingShareRunRequested = false;
            if (shareRun)
                seed = ShareReturn.TryTakeBattleSeed(_telemetry, _defense != null, _replay != null, seed);
            var level = Newbie.RandomLevel.Generate(seed);
            if (_watching)
            {
                // T014-B2b: the watched battle re-enacts the shadow recompute
                // live - the exact sim shape DefenseReplay.Replay drives
                // (default config, fusion off), so the watched settle equals
                // the recomputed verdict byte for byte.
                _sim = new BattleSim(seed, level);
            }
            else
            {
                // T011-B: every live battle runs with the mythic fusion armed -
                // the engine ctor stays default-off so seeded baselines ride
                // unchanged; the gate rhythm keeps waiting on the wave-window
                // adjudication.
                // T023-B: only the plain casual loop consults the grey gate;
                // destiny day-runs, defense challenges and share rematches pin
                // the build-uniform release so the same seed stays the same
                // battle on every device.
                string greyRelease = destiny || _defense != null || shareRun
                    ? null
                    : Sy.P02.Summon.SummonGreyGate.ResolveRelease();
                _sim = new BattleSim(seed, level, null, offerSize, mythicFusion: true, summonRelease: greyRelease);
            }
            if (destiny &amp;&amp; !_watching) _sim.MarkDestinyRun();   // T013-A: pure bookkeeping on the live path
            _view = gameObject.AddComponent&lt;BattleView&gt;();
            _view.Initialize(canvas, _sim);
            _view.SetSeedInfo((uint)seed, _sim.TotalWaves, level.ModifierName);
#if DEVELOPMENT_BUILD || UNITY_EDITOR
            // U133 / doc 12 section 6: dev-only one-tap bug report - the battle seed
            // is the reproduction key (a corpus seed replays the exact level).
            Sy.Quality.BugReportService.Game = "P02";
            Sy.Quality.BugReportService.Version = Application.version;
            Sy.Quality.BugReportService.SeedInfo = seed.ToString();
            Sy.Quality.BugReportService.ContextJson = () =&gt; P02QualityContext(_sim, level);
            Sy.Quality.BugReportButton.Attach(canvas.transform);
#endif
            if (_watching)
            {
                _view.SetSpectator(true);
                _view.SetReplayBadge(_replaySource);
            }
            // (telemetry resolved at the top of Awake - T015b-B moved it ahead of
```

```
// the seed chain so the shared-segment take can fire its event there)
_tutorial = new TutorialFlow(
    _sim,
    hint =&gt;
    {
        Debug.Log("[HINT] " + hint);
        _view.SetHint(hint);
    },
    step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
        new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
_view.SummonRequested += OnSummonRequested;
_view.RecycleRequested += OnRecycleRequested;
_sim.UnitRecycled += OnUnitRecycled;
_sim.StateChanged += OnStateChanged;
_sim.MythicFused += OnMythicFused;   // T011-B: live fusion telemetry
_sim.LifeRestored += OnLifeRestored;   // T011b-B2: heal flash + evt
_sim.GhostSpawned += OnGhostSpawned;   // T011b-B2: clone beat + evt
_sim.LeakBlocked += OnLeakBlocked;     // T011b-B2: intercept beat + evt
// T006-B2: rescue-ad traffic (clear_wave point, § ). The boot-registered
// AdCapPolicy is the same instance the suite's ShowRewarded consumes
// from, so the pre-check and the consumption stay consistent; without
// boot the fallback policy scopes the cap to this session.
if (!Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.AdCapPolicy&gt;(out _adCap))
    _adCap = new Sy.Platform.AdCapPolicy();
// T013-B2: the open-data friend board rides the platform suite; a
// direct scene entry without boot keeps it null and the settle-time
// submit stays a no-op (headless baselines untouched either way).
Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IOpenDataBoard&gt;(out _openData);
_sim.RescueOffered += OnRescueOffered;
_sim.RescueCleared += wave =&gt; _view.PlayRescueClearFlash(wave);
_view.RescueAcceptRequested += OnRescueAcceptRequested;
_view.RescueDeclineRequested += OnRescueDeclineRequested;
// T015-A1: shield lethal-guard traffic (§  point 1) - same wiring shape
// as the rescue pair; the sim keeps the two offers mutually exclusive.
_sim.ShieldOffered += OnShieldOffered;
_sim.ShieldBlocked += OnShieldBlocked;
_view.ShieldAcceptRequested += OnShieldAcceptRequested;
_view.ShieldDeclineRequested += OnShieldDeclineRequested;
// T015-A2: double-category traffic (§  points 4-5) - same three-piece
// shape as the crisis pairs; both points share the double 5/day cap.
_sim.SummonRerollOffered += OnRerollOffered;
_view.RerollAcceptRequested += OnRerollAcceptRequested;
_view.RerollDeclineRequested += OnRerollDeclineRequested;
// T015-A3: board_expand traffic (§  point 3) - the board-full pain
// moment opens the offer; the rescue category caps it (4/day shared
// with shield/clear_wave), same three-piece shape as the pairs above.
_sim.BoardExpandOffered += OnExpandOffered;
_view.ExpandAcceptRequested += OnExpandAcceptRequested;
_view.ExpandDeclineRequested += OnExpandDeclineRequested;
_view.SettleDoubleRequested += OnSettleDoubleRequested;
```

```
_view.ShareRequested += OnShareRequested;
_view.ClipShareRequested += OnClipShareRequested;
// T012-B: the wallet loads once per scene entry; the settle screen
// is its only surface (deposit at settle, redeem at the button).
_wallet = LuckyWallet.Load();
_view.ExchangeRequested += OnExchangeRequested;
// X813 slice A: real-clip audio wiring (G11 WorkerAudioRuntime
// paradigm). BGM rides the battle scene; SFX ride sim events with
// the null-safe law - a missing catalog keeps runs silent and all
// headless gates green. Spectator replays keep the same face.
// X816 slice C volume pass: battle BGM one mix-tier under the SFX
// layer (menu 0.55 / battle 0.6) so combat beats read through.
_audioHost = P02AudioHost.Ensure("bgm_battle", 0.6f);
_sim.UnitSummoned += _ =&gt; _audioHost.PlaySfx("sfx_unit_place", 0.9f);
_sim.UnitMerged += _ =&gt; _audioHost.PlaySfx("sfx_unit_merge", 0.9f);
_sim.UnitAttacked += _ =&gt; _audioHost.PlaySfx("sfx_unit_attack", 0.75f);
_sim.EnemyDamaged += _ =&gt; _audioHost.PlaySfx("sfx_enemy_hit", 0.7f);
_sim.GoldSummoned += _ =&gt; _audioHost.PlaySfx("sfx_gold", 0.85f);
_sim.WaveCleared += _ =&gt; _audioHost.PlaySfx("sfx_wave_win", 0.9f);
// X814 slice B: remaining sim-side beats (audio manifest rows 8/10/9/16/17).
_sim.EnemyKilled += _ =&gt; _audioHost.PlaySfx("sfx_enemy_die", 0.75f);
_sim.WaveStarted += _ =&gt; _audioHost.PlaySfx("sfx_alert", 0.8f);
_sim.MythicFused += (_, __) =&gt; _audioHost.PlaySfx("sfx_mythic_fuse", 1f);
_sim.StateChanged += s =&gt; { if (s == BattleSim.BattleState.Lost) _audioHost.PlaySfx("sfx_lose", 0.95f); }
_sim.RescueCleared += _ =&gt; _audioHost.PlaySfx("sfx_ad_reward", 0.9f);
// X817: T009 tap-reveal gesture closes its reserved beat - the view
// raises the tap on a cloaked foe, the sim flips it visible, and the
// EnemyRevealed payoff fires the reserved card-flip clip @0.8 (X816
// ruling: the beat was deferred until the gesture face existed).
_sim.EnemyRevealed += e =&gt;
{
    _audioHost.PlaySfx("sfx_card_flip", 0.8f);
    _view.PlayRevealFlash(e.Id);
};
_view.RevealRequested += enemy =&gt; _sim.TryRevealEnemy(enemy);
// X814 slice B: global click feedback on every battle-surface entry
// (summon keeps its dedicated place beat; main-menu tabs = next slice).
_view.RecycleRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.RescueAcceptRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.RescueDeclineRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ShieldAcceptRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ShieldDeclineRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.RerollAcceptRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.RerollDeclineRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ExpandAcceptRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ExpandDeclineRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.SettleDoubleRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ShareRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ClipShareRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_view.ExchangeRequested += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
```

```
            // T015-A4: consume the main screen's armed purple_pick grant on a
            // standard run only (fair play: destiny is the same battle for every
            // device, and challenge/replay must settle byte-identical - those
            // entries leave the arm pending for the next standard run).
            if (PendingPurplePick &amp;amp;&amp;amp; _defense == null &amp;amp;&amp;amp; _replay == null &amp;amp;&amp;amp; !destiny)
            {
                PendingPurplePick = false;
                if (_sim.ApplyPurplePick(out var granted, out var grantReason))
                    _view.SetHint("Purple " + NewbieConfig.GetUnit(granted.TypeId).Name + " joins your board!");
                else
                    Debug.Log("[T015-A4] purple_pick grant deferred: " + grantReason);
            }
        }
#if DEVELOPMENT_BUILD || UNITY_EDITOR
        /// &lt;summary&gt;U133 / doc 12 section 6: dev-only report context, fresh at tap
        /// time. The BR-*.json context block is what makes a user report
        /// mechanically reproducible (seed + live battle state).&lt;/summary&gt;
        static string P02QualityContext(BattleSim sim, Newbie.RandomLevel level)
        {
            if (sim == null) return "{}";
            return "{\"state\":\"" + sim.State + "\",\"wave\":" + sim.Wave
                + ",\"totalWaves\":" + sim.TotalWaves
                + ",\"kills\":" + sim.Kills
                + ",\"crystals\":" + sim.Crystals
                + ",\"merges\":" + sim.MergeCount
                + ",\"rescues\":" + sim.RescueCount
                + ",\"summons\":" + sim.SummonCount
                + ",\"units\":" + (sim.Units != null ? sim.Units.Count : 0)
                + ",\"battleSec\":" + sim.TimedElapsed.ToString("F1", System.Globalization.CultureInfo.InvariantCulture)
                + ",\"modifier\":\"" + (level != null ? level.ModifierName : "") + "\"}";
        }
#endif
        void Update()
        {
            if (_watching)
            {
                // T014-B2b: fixed-step watch loop - real frames only pace the
                // playback; the executed steps, their order and the guard
                // indexes match DefenseReplay.ShadowPolicy exactly, so the
                // watched settle is byte-identical to the core recompute.
                _watchAccum += Time.deltaTime;
                while (_watchAccum &gt;= WatchStepSeconds
                       &amp;&amp; _sim.State != BattleSim.BattleState.Won
                       &amp;&amp; _sim.State != BattleSim.BattleState.Lost
                       &amp;&amp; _watchTime &lt; WatchMaxSeconds
                       &amp;&amp; _watchGuard++ &lt; 20000)
                {
                    _watchAccum -= WatchStepSeconds;
                    DefenseReplay.ShadowStep(_sim, _watchGuard);
                    _sim.Tick(WatchStepSeconds);
```

```
            _watchTime += WatchStepSeconds;
        }
        if (_sim.State != BattleSim.BattleState.Won
            &amp;&amp; _sim.State != BattleSim.BattleState.Lost
            &amp;&amp; (_watchGuard &gt;= 20000 || _watchTime &gt;= WatchMaxSeconds))
        {
            // Safety valve: a seed the shadow cannot settle inside the
            // policy window ends the watch honestly instead of freezing.
            _watching = false;
            _view.ShowSettle(false, _sim.Wave, _sim.Kills, 0, _wallet.Balance, false);
            _view.ShowSettleDefense(
                DefenseReplay.ComparisonLine(DefenseReplay.Verdict.None, null, null), null);
            TrackReplayEnd("none");
        }
    }
    else
    {
        _sim.Tick(Time.deltaTime);
    }
    _view.SyncView(_sim);
    if (_sim.Wave != _lastWave) { _lastWave = _sim.Wave; _view.SetWave(_lastWave); }
    if (_sim.Life != _lastLife) { _lastLife = _sim.Life; _view.SetLife(_lastLife); }
    if (_sim.Crystals != _lastCrystals) { _lastCrystals = _sim.Crystals; _view.SetCrystals(_lastCrystals, _sim.F
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
}
void OnSummonRequested()
{
    // Task T003: the odds popup gates only the first summon of a session;
    // the sim itself stays unblocked (headless checks drive it directly).
    if (!_oddsAcknowledged)
    {
        _oddsAcknowledged = true;
        _view.ShowOddsPopup(DoSummon);
        return;
    }
    DoSummon();
}
void DoSummon()
{
    // T005: both failure modes point at the recycle valve (English strings
    // per the source rule; full i18n lands with X023).
    if (!_sim.TrySummon(out _, out var reason))
        _view.SetHint(reason == "no_crystals" ? "Not enough crystals - recycle a white unit"
            : reason == "board_full" ? "Board full - recycle a white unit to free a slot"
            : "Battle over");
}
void OnRecycleRequested()
{
    var unit = _sim.FindRecyclableUnit();
    if (unit == null)
```

```
            "slow" =&gt; Role.Slow,
            "pierce" =&gt; Role.Pierce,
            "burst" =&gt; Role.Burst,
            "mythic" =&gt; Role.Mythic,
            "control" =&gt; Role.Control,
            "dot" =&gt; Role.Dot,
            "heal" =&gt; Role.Heal,
            "economy" =&gt; Role.Economy,
            "summon" =&gt; Role.Summon,
            "shield" =&gt; Role.Shield,
            _ =&gt; Role.Aoe,
        };
        static int AsInt(Dictionary&lt;string, object&gt; node, string key)
        {
            if (!node.TryGetValue(key, out var v)) return 0;
            return v switch
            {
                long l =&gt; (int)l,
                double d =&gt; (int)d,
                _ =&gt; 0,
            };
        }
        static float AsFloat(Dictionary&lt;string, object&gt; node, string key)
        {
            if (!node.TryGetValue(key, out var v)) return 0f;
            return v switch
            {
                long l =&gt; (float)l,
                double d =&gt; (float)d,
                _ =&gt; 0f,
            };
        }
    }
}
// SummonGreyGate.cs
using UnityEngine;
namespace Sy.P02.Summon
{
    /// &lt;summary&gt;
    /// T023-B local grey gate: a persistent device bucket (0-99, rolled once,
    /// PlayerPrefs authority - red line 12) fixes the treatment cohort for the
    /// dormant weight candidate. The grey channel is double-gated: the cohort
    /// only receives the candidate when BOTH the reviewed build arms the
    /// channel (the arm flips in a reviewed update, after the bot sweep pins
    /// real v1.1 values) AND the device falls inside the treatment percent.
    /// A full release flip (ActiveRelease off the baseline) overrides grey
    /// entirely - a reviewed rollout serves everyone. Competitive surfaces
    /// (destiny day-run, defense challenge, spectator watch, share rematch)
    /// never consult this gate: they pin the build-uniform Live() release so
    /// the same seed stays the same battle on every device.
```

```
    /// &lt;/summary&gt;
    public static class SummonGreyGate
    {
        public const string BucketKey = "p02_summon_grey_bucket";
        public const int TreatmentPercent = 5;
        // T025 submission arm: SY_SUBMISSION is injected per-build by
        // BuildWeixin's extraScriptingDefines and nowhere else, so editor,
        // rehearsal and every gate build stay unarmed (zero drift) while the
        // shipped package alone arms the 5% cohort (C4: the arm flips with
        // the reviewed submission version, never at runtime).
#if SY_SUBMISSION
        static bool _greyArmed = true;
#else
        static bool _greyArmed = false;
#endif
        /// &lt;summary&gt;Grey channel arm. False in editor/rehearsal builds (the
        /// dormant candidate stays unreachable - T023-A law); the submission
        /// package flips it true through the compiled SY_SUBMISSION face.
        /// Static readonly pattern: tests flip it through the internal setter
        /// (SetReleaseForTests convention).&lt;/summary&gt;
        public static bool GreyArmed =&gt; _greyArmed;
        /// &lt;summary&gt;Editor/rehearsal flip point; live builds flip the default
        /// in a reviewed update, never at runtime.&lt;/summary&gt;
        internal static void SetArmedForTests(bool armed) { _greyArmed = armed; }
        /// &lt;summary&gt;Lazy persistent device bucket: the first call rolls 0-99
        /// once, then the device keeps its cohort forever (DestinyGate roll
        /// pattern - one-time PlayerPrefs pick, stable across sessions).&lt;/summary&gt;
        public static int Bucket()
        {
            int bucket = PlayerPrefs.GetInt(BucketKey, -1);
            if (bucket &lt; 0 || bucket &gt; 99)
            {
                bucket = Random.Range(0, 100);
                PlayerPrefs.SetInt(BucketKey, bucket);
            }
            return bucket;
        }
        /// &lt;summary&gt;Pure cohort law (gate-assertable): armed &amp;&amp; bucket inside
        /// [0, TreatmentPercent) is treatment; anything else is control. A
        /// corrupt out-of-range bucket reads as control, never treatment.&lt;/summary&gt;
        public static bool IsTreatmentFor(int bucket, bool armed)
            =&gt; armed &amp;&amp; bucket &gt;= 0 &amp;&amp; bucket &lt; TreatmentPercent;
        /// &lt;summary&gt;Pure release resolution: a full rollout (ActiveRelease off
        /// the baseline) wins outright - grey only decides while the live
        /// release is still the baseline, so a reviewed flip reaches every
        /// device regardless of cohort.&lt;/summary&gt;
        public static string ResolveReleaseFor(int bucket, bool armed)
        {
            var active = SummonWeightTable.ActiveRelease;
            if (active != SummonWeightTable.BaselineRelease) return active;
```

```
            return IsTreatmentFor(bucket, armed)
                ? SummonWeightTable.CandidateRelease
                : SummonWeightTable.BaselineRelease;
        }
        /// &lt;summary&gt;Release a casual run rolls under (this device's cohort).&lt;/summary&gt;
        public static string ResolveRelease() =&gt; ResolveReleaseFor(Bucket(), _greyArmed);
        /// &lt;summary&gt;Config every casual consumer reads (battle pool + odds
        /// disclosure): one resolution point, so the published odds can only
        /// ever equal the odds this device's casual battles roll (T003
        /// disclosure law).&lt;/summary&gt;
        public static SummonPoolConfig LiveConfig() =&gt; SummonWeightTable.Profile(ResolveRelease());
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(BucketKey);
            _greyArmed = false;
        }
    }
}
// SummonPool.cs
using System;
using System.Collections.Generic;
using System.Globalization;
using Sy.Core;
using Sy.P02.Newbie;
namespace Sy.P02.Summon
{
    /// &lt;summary&gt;
    /// Deterministic PRNG (xorshift64*): the algorithm is owned by this file, so one
    /// seed replays identically on every runtime (mono editor, il2cpp players, dotnet
    /// tools) - the prerequisite for the T013 locked-config seeded runs, where the
    /// System.Random implementation is not guaranteed to match across runtimes.
    /// &lt;/summary&gt;
    public sealed class SeededRandom
    {
        ulong _state;
        public SeededRandom(ulong seed)
        {
            _state = seed == 0 ? 0x9E3779B97F4A7C15ul : seed;
        }
        /// &lt;summary&gt;Raw 64-bit step (xorshift64*, period 2^64-1).&lt;/summary&gt;
        public ulong NextUInt64()
        {
            ulong x = _state;
            x ^= x &gt;&gt; 12;
            x ^= x &lt;&lt; 25;
            x ^= x &gt;&gt; 27;
            _state = x;
            return x * 0x2545F4914F6CDD1Dul;
        }
        /// &lt;summary&gt;Unbiased integer in [0, maxExclusive) via modulo rejection.&lt;/summary&gt;
```

```
        public int NextInt(int maxExclusive)
        {
            if (maxExclusive &lt;= 1) return 0;
            ulong range = (ulong)maxExclusive;
            ulong limit = ulong.MaxValue - (ulong.MaxValue % range);
            while (true)
            {
                ulong v = NextUInt64();
                if (v &lt; limit) return (int)(v % range);
            }
        }
    }
    /// &lt;summary&gt;
    /// Immutable summon odds (task T003 / numeric pack § ). Two sources build it:
    /// FromDefaults (embedded slice, live until the hot-update pipeline lands) and
    /// FromJson (cfg_summon_weight.json, validated by ConfigValidator - disclosed:true,
    /// both weight tables sum to 100, ad pool white = 0). Disclosure strings are
    /// generated from these same weights, so published odds can only ever equal the
    /// odds the pool actually rolls (   =   , enforced by SummonCheck).
    /// &lt;/summary&gt;
    public sealed class SummonPoolConfig
    {
        public int[] MainWeights;          // indexed by Tier (White..Gold)
        public int[] AdWeights;
        public int FirstGoldPity;          // 0 = disabled
        public bool Disclosed;
        public string ProbabilitiesPrecision;
        public static SummonPoolConfig FromDefaults()
        {
            return new SummonPoolConfig
            {
                MainWeights = new[]
                {
                    NewbieConfig.WeightWhite, NewbieConfig.WeightBlue,
                    NewbieConfig.WeightPurple, NewbieConfig.WeightGold,
                },
                AdWeights = new[]
                {
                    NewbieConfig.AdWeightWhite, NewbieConfig.AdWeightBlue,
                    NewbieConfig.AdWeightPurple, NewbieConfig.AdWeightGold,
                },
                FirstGoldPity = NewbieConfig.FirstGoldPity,
                Disclosed = true,
                ProbabilitiesPrecision = NewbieConfig.ProbabilitiesPrecision,
            };
        }
        /// &lt;summary&gt;Parse cfg_summon_weight.json content (MiniJson: long/double/bool/string).
        /// T023-A: the versioned shape carries a profiles map plus activeProfile -
        /// the active profile is returned; the legacy flat shape (v1) still parses
        /// so shipped builds and old authoring slices keep working.&lt;/summary&gt;
```

```
public static SummonPoolConfig FromJson(string json)
{
    var root = MiniJson.Deserialize(json) as Dictionary&lt;string, object&gt;;
    if (root == null) return null;
    if (root.TryGetValue("profiles", out var p) &amp;&amp; p is Dictionary&lt;string, object&gt; profiles)
    {
        string active = root.TryGetValue("activeProfile", out var a) ? a as string : null;
        if (active == null
            || !profiles.TryGetValue(active, out var node)
            || node is not Dictionary&lt;string, object&gt; profile) return null;
        return FromProfileNode(profile);
    }
    return FromProfileNode(root);
}
/// &lt;summary&gt;Parse one profile node (weights/adPoolWeights/firstGoldPity);
/// shared by the versioned profiles map and the legacy flat root.&lt;/summary&gt;
public static SummonPoolConfig FromProfileNode(Dictionary&lt;string, object&gt; root)
{
    if (root == null) return null;
    var weights = root.TryGetValue("weights", out var w) ? ReadTierTable(w) : null;
    var adWeights = root.TryGetValue("adPoolWeights", out var a2) ? ReadTierTable(a2) : null;
    if (weights == null || adWeights == null) return null;
    return new SummonPoolConfig
    {
        MainWeights = weights,
        AdWeights = adWeights,
        FirstGoldPity = AsInt(root, "firstGoldPity"),
        Disclosed = root.TryGetValue("disclosed", out var d) &amp;&amp; d is bool b &amp;&amp; b,
        ProbabilitiesPrecision = root.TryGetValue("probabilitiesPrecision", out var p2) ? p2 as string : null,
    };
}
static int[] ReadTierTable(object node)
{
    if (node is not Dictionary&lt;string, object&gt; table) return null;
    return new[]
    {
        AsInt(table, "white"), AsInt(table, "blue"),
        AsInt(table, "purple"), AsInt(table, "gold"),
    };
}
static int AsInt(Dictionary&lt;string, object&gt; node, string key)
{
    if (!node.TryGetValue(key, out var v)) return 0;
    return v switch
    {
        long l =&gt; (int)l,
        double d =&gt; (int)d,
        _ =&gt; 0,
    };
}
```

```
    int Total(int[] weights)
    {
        int sum = 0;
        for (int i = 0; i &lt; weights.Length; i++) sum += weights[i];
        return sum;
    }
    /// &lt;summary&gt;Disclosed percent string ("70.00%") generated from the live weights.&lt;/summary&gt;
    public string MainPercent(Tier tier) =&gt; Percent(MainWeights[(int)tier], Total(MainWeights));
    public string AdPercent(Tier tier) =&gt; Percent(AdWeights[(int)tier], Total(AdWeights));
    static string Percent(int weight, int total)
    {
        double pct = total &gt; 0 ? weight * 100.0 / total : 0.0;
        return pct.ToString("F2", CultureInfo.InvariantCulture) + "%";
    }
    /// &lt;summary&gt;Published soft-pity rule line; the threshold follows the config so a
    /// hot-tune (e.g. pity 6 -&gt; 4) updates every disclosure surface automatically.&lt;/summary&gt;
    public string PityRuleText()
    {
        return "Lucky accumulation: the " + Ordinal(FirstGoldPity)
             + " summon guarantees a Gold revelation.";
    }
    static string Ordinal(int n)
    {
        if (n &lt;= 0) return n.ToString();
        int mod100 = n % 100;
        if (mod100 &gt;= 11 &amp;&amp; mod100 &lt;= 13) return n + "th";
        switch (n % 10)
        {
            case 1: return n + "st";
            case 2: return n + "nd";
            case 3: return n + "rd";
            default: return n + "th";
        }
    }
}
/// &lt;summary&gt;
/// Quality-tier summon pool (task T003). Main pool = free/crystal summons;
/// ad pool = the "never rolls white" reroll pool for ad point #5. First-gold
/// soft pity: every draw the player sees (main, ad, or scripted/external) counts;
/// until the first gold lands, the pity-th draw is forced Gold. After the first
/// gold the pool rolls pure weights forever, so the 100k-draw distribution stays
/// within tolerance of the published table.
/// &lt;/summary&gt;
public sealed class SummonPool
{
    readonly SummonPoolConfig _config;
    readonly SeededRandom _rng;
    int _drawCount;
    bool _goldSeen;
    public SummonPool(SummonPoolConfig config, ulong seed)
```

```
        {
            _config = config;
            _rng = new SeededRandom(seed);
        }
        public SummonPoolConfig Config =&gt; _config;
        public int DrawCount =&gt; _drawCount;
        public bool GoldSeen =&gt; _goldSeen;
        public int MainWeight(Tier tier) =&gt; _config.MainWeights[(int)tier];
        public int AdWeight(Tier tier) =&gt; _config.AdWeights[(int)tier];
        public Tier DrawMain(out bool pityForced) =&gt; Draw(_config.MainWeights, out pityForced);
        public Tier DrawAd(out bool pityForced) =&gt; Draw(_config.AdWeights, out pityForced);
        Tier Draw(int[] weights, out bool pityForced)
        {
            pityForced = false;
            _drawCount++;
            if (_config.FirstGoldPity &gt; 0 &amp;&amp; !_goldSeen &amp;&amp; _drawCount &gt;= _config.FirstGoldPity)
            {
                _goldSeen = true;
                pityForced = true;
                return Tier.Gold;
            }
            var tier = Pick(weights);
            if (tier == Tier.Gold) _goldSeen = true;
            return tier;
        }
        /// &lt;summary&gt;Count a scripted/synthetic draw (tutorial overrides) toward the pity
        /// so the published "Nth summon" counts every summon the player performs.&lt;/summary&gt;
        public void NotifyExternalDraw(Tier tier)
        {
            _drawCount++;
            if (tier == Tier.Gold) _goldSeen = true;
        }
        Tier Pick(int[] weights)
        {
            int total = 0;
            for (int i = 0; i &lt; weights.Length; i++) total += weights[i];
            if (total &lt;= 0) return Tier.White;
            int roll = _rng.NextInt(total);
            int acc = 0;
            for (int i = 0; i &lt; weights.Length; i++)
            {
                acc += weights[i];
                if (roll &lt; acc) return (Tier)i;
            }
            return (Tier)(weights.Length - 1);
        }
    }
}
// SummonWeightTable.cs
using System.Collections.Generic;
```

```
using Sy.Core;
using Sy.P02.Newbie;
namespace Sy.P02.Summon
{
    /// &lt;summary&gt;
    /// T023-A weight hot-tune channel, local edition (red line 12: remote
    /// hot-update is permanently banned, so profiles ship embedded and the
    /// release flips with a platform review update - C4 audit "config ships
    /// embedded, review-gated flips"). One profile = one complete
    /// SummonPoolConfig slice; a dormant candidate stays unreachable until
    /// ActiveRelease is bumped in a reviewed build. This layer guarantees:
    /// live values byte-equal the published v1.0 table, every profile obeys
    /// the gacha red lines (disclosed / sums / ad-pool white = 0), and an
    /// unknown release id falls back to the baseline so the pool never
    /// dead-ends. Local grey testing (seed bucket + config pin) is the
    /// T023-B slice.
    /// &lt;/summary&gt;
    public static class SummonWeightTable
    {
        public const string BaselineRelease = "v1.0";
        public const string CandidateRelease = "v1.1";
        static string _activeRelease = BaselineRelease;
        /// &lt;summary&gt;Live release id; flips only inside a reviewed build
        /// (OP01 update channel). Static readonly pattern: a const compared
        /// against a literal collapses and trips CS0162 zero-warning gates.&lt;/summary&gt;
        public static string ActiveRelease =&gt; _activeRelease;
        /// &lt;summary&gt;Editor-only flip point for the release rehearsal
        /// (UnitTable SetReleaseForTests pattern); live builds never call it.&lt;/summary&gt;
        internal static void SetActiveReleaseForTests(string id) { _activeRelease = id; }
        /// &lt;summary&gt;The config every runtime consumer reads (BattleSim pool,
        /// MainUI disclosure table): one switch point for the whole game, so
        /// published odds can only ever equal the rolled odds.&lt;/summary&gt;
        public static SummonPoolConfig Live() =&gt; Profile(_activeRelease);
        static readonly Dictionary&lt;string, SummonPoolConfig&gt; _sweepProfiles =
            new Dictionary&lt;string, SummonPoolConfig&gt;();
        /// &lt;summary&gt;Editor-only sweep registration (T023-C): the bot
        /// parameter-sweep gate injects candidate grids under pseudo
        /// release ids; BattleSim pins those ids the same way it pins real
        /// releases. Live builds never call this and the registry stays
        /// empty, so Profile() is byte-stable for every shipped id.&lt;/summary&gt;
        internal static void RegisterProfileForTests(string id, SummonPoolConfig cfg)
        {
            if (string.IsNullOrEmpty(id) || cfg == null) return;
            _sweepProfiles[id] = cfg;
        }
        /// &lt;summary&gt;Editor-only counterpart of RegisterProfileForTests;
        /// the sweep gate unregisters every arm it registered.&lt;/summary&gt;
        internal static void UnregisterProfileForTests(string id)
        {
            _sweepProfiles.Remove(id);
```

```
        }
        /// &lt;summary&gt;Embedded profile slice; unknown ids fall back to the
        /// baseline (a bad flip can never brick the pool).&lt;/summary&gt;
        public static SummonPoolConfig Profile(string id)
        {
            if (_sweepProfiles.Count &gt; 0 &amp;&amp; _sweepProfiles.TryGetValue(id, out var sweep))
                return sweep;
            if (id == CandidateRelease) return Candidate();
            return SummonPoolConfig.FromDefaults();
        }
        /// &lt;summary&gt;Authoring chain for the gate: read one profile node out
        /// of cfg_summon_weight.json content (MiniJson long/double/bool).
        /// Returns null when the table or the requested profile is missing.&lt;/summary&gt;
        public static SummonPoolConfig FromJsonProfile(string json, string id)
        {
            if (string.IsNullOrEmpty(json)) return null;
            var root = MiniJson.Deserialize(json) as Dictionary&lt;string, object&gt;;
            if (root == null) return null;
            if (!root.TryGetValue("profiles", out var p)
                || p is not Dictionary&lt;string, object&gt; profiles) return null;
            if (!profiles.TryGetValue(id, out var node)
                || node is not Dictionary&lt;string, object&gt; profile) return null;
            return SummonPoolConfig.FromProfileNode(profile);
        }
        /// &lt;summary&gt;The dormant candidate slice, calibrated by the T023-C bot
        /// sweep (SummonSweepCheck X107, 4 arms x 8 seeds: winner arm d keeps
        /// P50=241.1s inside the 240s+/-20s window at the highest epic+gold
        /// share, 8/8 merger-bot wins). Sums stay 100 and the ad pool keeps
        /// white = 0 - the gacha red lines hold for every profile, live or
        /// dormant. SW4 pins this slice to the sweep winner.&lt;/summary&gt;
        static SummonPoolConfig Candidate()
        {
            return new SummonPoolConfig
            {
                MainWeights = new[] { 67, 22, 10, 1 },
                AdWeights = new[] { 0, 64, 30, 6 },
                FirstGoldPity = NewbieConfig.FirstGoldPity,
                Disclosed = true,
                ProbabilitiesPrecision = NewbieConfig.ProbabilitiesPrecision,
            };
        }
    }
}
// BattlePackageTests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Battle;
using UnityEngine;
namespace Sy.P02.Tests
{
```

```
/// &lt;summary&gt;
/// com.sy.battle NUnit mirror of BattleCheck (T001d): loop matrix, timed wave
/// schedule, waypoint math + stress budget, pool zero-GC and degrade policy.
/// &lt;/summary&gt;
[TestFixture]
public sealed class BattlePackageTests
{
    sealed class RecordStrategy : IRunStrategy
    {
        public int EnterCount, TickCount, ExitCount;
        public string Id =&gt; "record";
        public void Enter(BattleContext ctx) =&gt; EnterCount++;
        public void Tick(float dt) =&gt; TickCount++;
        public void Exit() =&gt; ExitCount++;
    }
    [Test]
    public void LoopTransitionMatrixLocked()
    {
        var ctx = new BattleContext { Seed = 1, CfgVersion = "t" };
        var s = new RecordStrategy();
        var loop = new BattleLoop(s);
        // Prepare ignores everything except Start.
        loop.Tick(0.1f);
        loop.Settle();
        loop.EnterBossWave();
        Assert.AreEqual(BattlePhase.Prepare, loop.Phase);
        loop.Start(ctx);
        Assert.AreEqual(BattlePhase.Fighting, loop.Phase);
        Assert.AreEqual(1, s.EnterCount);
        // No mid-run restart; Tick drives strategy.
        loop.Tick(0.1f);
        loop.Start(ctx);
        Assert.AreEqual(BattlePhase.Fighting, loop.Phase);
        Assert.AreEqual(1, s.EnterCount);
        Assert.AreEqual(1, s.TickCount);
        // BossWave only from Fighting; Tick keeps driving.
        loop.EnterBossWave();
        Assert.AreEqual(BattlePhase.BossWave, loop.Phase);
        loop.EnterBossWave();
        loop.Tick(0.1f);
        Assert.AreEqual(BattlePhase.BossWave, loop.Phase);
        Assert.AreEqual(2, s.TickCount);
        // Settle only from combat phases; End ignores everything; restart allowed.
        loop.Settle();
        Assert.AreEqual(BattlePhase.Settle, loop.Phase);
        loop.Tick(0.1f);
        loop.Start(ctx);
        loop.EnterBossWave();
        Assert.AreEqual(BattlePhase.Settle, loop.Phase);
        loop.Finish();
```

```
        Assert.AreEqual(BattlePhase.End, loop.Phase);
        Assert.AreEqual(1, s.ExitCount);
        loop.Finish();
        Assert.AreEqual(1, s.ExitCount, "double Finish must not exit twice");
        loop.Start(ctx);
        Assert.AreEqual(BattlePhase.Fighting, loop.Phase);
        Assert.AreEqual(2, s.EnterCount);
    }
    [Test]
    public void TimedWaveScheduleAndBounds()
    {
        var cfg = TimedWaveConfig.BuildDefault(16);
        Assert.AreEqual(16, cfg.Waves.Count);
        Assert.AreEqual(240f, cfg.DurationSeconds, 0.001f);
        Assert.AreEqual(0f, cfg.Waves[0].StartSec, 0.001f);
        Assert.AreEqual(225f, cfg.Waves[15].StartSec, 0.001f);
        Assert.AreEqual(15, TimedWaveConfig.BuildDefault(10).Waves.Count, "clamped up to 15");
        Assert.AreEqual(18, TimedWaveConfig.BuildDefault(25).Waves.Count, "clamped down to 18");
        var run = new TimedWaveRun(cfg);
        var started = new List&lt;WavePlan&gt;();
        var bosses = new List&lt;WavePlan&gt;();
        run.OnWaveStarted += started.Add;
        run.OnBossWave += bosses.Add;
        run.Enter(new BattleContext { Seed = 7, CfgVersion = "t" });
        int guard = 0;
        while (!run.IsFinished &amp;&amp; guard++ &lt; 10000) run.Tick(0.05f);
        Assert.IsTrue(run.IsFinished);
        Assert.AreEqual(16, run.WavesFired);
        Assert.AreEqual(16, started.Count);
        Assert.AreEqual(3, bosses.Count);
        Assert.AreEqual(0, started[0].Index);
        Assert.AreEqual(15, started[15].Index);
    }
    [Test]
    public void WaypointMathIsExactAndStressStaysInBudget()
    {
        var path = PathData.Build(new Vector2(0, 0), new Vector2(10, 0), new Vector2(10, 10));
        Assert.AreEqual(20f, path.TotalLength, 0.001f);
        var s = WaypointResolver.Restart(path);
        WaypointResolver.Advance(path, ref s, 5f, 1f);
        Assert.Less(Vector2.Distance(s.Pos, new Vector2(5, 0)), 0.001f);
        WaypointResolver.Advance(path, ref s, 5f, 1f);
        Assert.Less(Vector2.Distance(s.Pos, new Vector2(10, 0)), 0.001f);
        Assert.AreEqual(1, s.Seg);
        WaypointResolver.Advance(path, ref s, 5f, 1f);
        WaypointResolver.Advance(path, ref s, 5f, 1f);
        Assert.IsTrue(s.Finished);
        Assert.Less(Vector2.Distance(s.Pos, new Vector2(10, 10)), 0.001f);
        WaypointResolver.Advance(path, ref s, 5f, 1f);
        Assert.Less(Vector2.Distance(s.Pos, new Vector2(10, 10)), 0.001f, "finished movers must not move");
```

```
        var states = new MoveState[200];
        for (int i = 0; i &lt; states.Length; i++) states[i] = WaypointResolver.Restart(path);
        var sw = System.Diagnostics.Stopwatch.StartNew();
        for (int t = 0; t &lt; 1000; t++)
            for (int i = 0; i &lt; states.Length; i++)
                WaypointResolver.Advance(path, ref states[i], 5f, 0.05f);
        sw.Stop();
        int finished = 0;
        for (int i = 0; i &lt; states.Length; i++)
        {
            if (states[i].Finished) finished++;
            Assert.IsFalse(float.IsNaN(states[i].Pos.x) || float.IsNaN(states[i].Pos.y));
        }
        Assert.AreEqual(200, finished);
        Assert.Less((float)sw.Elapsed.TotalMilliseconds, 200f,
            "200x1000 advances must stay far under one frame per tick budget");
    }
    [Test]
    public void PoolZeroGcRoundsAndDegradeOldest()
    {
        var pool = new BattlePool&lt;object&gt;(() =&gt; new object(), 1000);
        Assert.AreEqual(1000, pool.CreatedCount);
        var borrowed = new object[1000];
        var distinct = new HashSet&lt;object&gt;();
        for (int i = 0; i &lt; 1000; i++)
        {
            borrowed[i] = pool.Borrow(out _);
            distinct.Add(borrowed[i]);
        }
        Assert.AreEqual(1000, distinct.Count);
        Assert.AreEqual(1000, pool.ActiveCount);
        for (int i = 0; i &lt; 1000; i++) pool.Return(borrowed[i]);
        for (int i = 0; i &lt; 1000; i++) borrowed[i] = pool.Borrow(out _);
        System.GC.Collect();
        int g0 = System.GC.CollectionCount(0), g1 = System.GC.CollectionCount(1), g2 = System.GC.CollectionCount(2);
        for (int round = 0; round &lt; 3; round++)
        {
            for (int i = 0; i &lt; 1000; i++) pool.Return(borrowed[i]);
            for (int i = 0; i &lt; 1000; i++) borrowed[i] = pool.Borrow(out _);
        }
        Assert.AreEqual(0, System.GC.CollectionCount(0) - g0, "gen0 collections during rounds");
        Assert.AreEqual(0, System.GC.CollectionCount(1) - g1, "gen1 collections during rounds");
        Assert.AreEqual(0, System.GC.CollectionCount(2) - g2, "gen2 collections during rounds");
        for (int i = 0; i &lt; 1000; i++) pool.Return(borrowed[i]);
        for (int i = 0; i &lt; 1000; i++) borrowed[i] = pool.Borrow(out _);
        var degradedSet = new HashSet&lt;object&gt;();
        for (int i = 0; i &lt; 50; i++)
        {
            var item = pool.Borrow(out bool degraded);
            Assert.IsTrue(degraded);
```

```
                degradedSet.Add(item);
            }
            Assert.AreEqual(50, pool.DegradeCount);
            Assert.AreEqual(50, degradedSet.Count, "degrade must rotate across distinct instances");
            Assert.AreEqual(1000, pool.CreatedCount, "overflow never grows the pool");
            for (int i = 0; i &lt; 1000; i++) pool.Return(borrowed[i]);
            pool.Return(new object());      // unknown: ignored
            pool.Return(borrowed[0]);       // double: ignored
            Assert.AreEqual(0, pool.ActiveCount);
            var next = pool.Borrow(out bool dg2);
            Assert.IsFalse(dg2);
            Assert.AreSame(borrowed[999], next, "borrow after return is LIFO");
        }
        [Test]
        public void RunResultBuilderAccumulates()
        {
            var b = new RunResultBuilder();
            b.AddKills(3);
            b.AddKill();
            b.AddCoins(120);
            b.AddCoins(30);
            b.SetDuration(240f);
            b.SetWaveReached(16);
            b.AddDrop(101);
            b.AddDrop(102);
            b.AddUnlockCard("star_frost");
            var result = b.Build();
            Assert.AreEqual(4, result.Kills);
            Assert.AreEqual(150, result.Coins);
            Assert.AreEqual(240f, result.DurationSeconds, 0.001f);
            Assert.AreEqual(16, result.WaveReached);
            CollectionAssert.AreEqual(new[] { 101, 102 }, result.Drops);
            CollectionAssert.AreEqual(new[] { "star_frost" }, result.UnlockCards);
        }
    }
}
// BoardPackageTests.cs
using NUnit.Framework;
using Sy.Board;
using UnityEngine;
namespace Sy.P02.Tests
{
    /// &lt;summary&gt;
    /// com.sy.board NUnit mirror of BoardCheck (T002): model, geometry, placement
    /// rule, three-merge, scanner, zero-GC loops.
    /// &lt;/summary&gt;
    [TestFixture]
    public sealed class BoardPackageTests
    {
        [Test]
```

```
public void BoardModelAndOccupancy()
{
    var board = new GridBoard();
    Assert.AreEqual(5, board.Cols);
    Assert.AreEqual(8, board.Rows);
    Assert.AreEqual(40, board.CellCount);
    Assert.AreEqual(6, new GridBoard(6, 7).Cols);
    Assert.IsTrue(board.Place(1, 1, 7, 1));
    Assert.IsFalse(board.Place(1, 1, 9, 1), "second place rejected");
    Assert.IsTrue(board.TryGetOccupant(1, 1, out int id, out int level));
    Assert.AreEqual(7, id);
    Assert.AreEqual(1, level);
    Assert.IsTrue(board.Remove(1, 1));
    Assert.IsFalse(board.Remove(1, 1));
    Assert.IsTrue(board.TryStack(2, 2, 5, 1));
    Assert.IsTrue(board.TryStack(2, 2, 5, 1));
    Assert.IsFalse(board.TryStack(2, 2, 9, 1), "different id rejected");
    Assert.IsFalse(board.TryStack(2, 2, 5, 2), "different level rejected");
    Assert.IsTrue(board.TryStack(2, 2, 5, 1));
    Assert.IsFalse(board.TryStack(2, 2, 5, 1), "cap 3 enforced");
    Assert.AreEqual(3, board.GetCell(2, 2).StackCount);
}
[Test]
public void WorldCellMappingRoundtrips()
{
    var board = new GridBoard(5, 8) { CellSize = new Vector2(128f, 128f), OriginWorld = new Vector2(64f, 128f) }
    Assert.Less(Vector2.Distance(board.CellToWorld(0, 0), new Vector2(128f, 192f)), 0.001f);
    for (int c = 0; c &lt; board.Cols; c++)
        for (int r = 0; r &lt; board.Rows; r++)
        {
            var cell = board.WorldToCell(board.CellToWorld(c, r));
            Assert.AreEqual(c, cell.Col);
            Assert.AreEqual(r, cell.Row);
        }
    Assert.AreEqual(-1, board.WorldToCell(new Vector2(-500f, -500f)).Col);
    Assert.AreEqual(-1, board.WorldToCell(new Vector2(64f + 5.5f * 128f, 192f)).Col);
}
[Test]
public void PlaceRuleTableLookup()
{
    var board = new GridBoard();
    var rule = new TowerPlaceRule(board);
    board.SetType(0, 0, CellType.PathBlocked);
    board.SetType(1, 0, CellType.Spawn);
    board.SetType(2, 0, CellType.Exit);
    Assert.IsFalse(rule.CanPlace(0, 0, 1));
    Assert.IsFalse(rule.CanPlace(1, 0, 1));
    Assert.IsFalse(rule.CanPlace(2, 0, 1));
    Assert.IsTrue(rule.CanPlace(3, 0, 1));
    Assert.IsFalse(rule.CanPlace(-1, 0, 1));
```

```
        board.Place(4, 4, 5, 1);
        Assert.IsTrue(rule.CanPlace(4, 4, 5));
        Assert.IsFalse(rule.CanPlace(4, 4, 9));
        board.TryStack(4, 4, 5, 1);
        board.TryStack(4, 4, 5, 1);
        Assert.IsFalse(rule.CanPlace(4, 4, 5), "full stack rejects drops");
    }
    [Test]
    public void ThreeMergeLevelsUpInPlace()
    {
        var board = new GridBoard();
        board.TryStack(0, 0, 5, 1);
        board.TryStack(0, 0, 5, 1);
        Assert.IsFalse(MergeResolver.TryMergeAt(board, 0, 0));
        board.TryStack(0, 0, 5, 1);
        Assert.IsTrue(MergeResolver.TryMergeAt(board, 0, 0));
        ref var cell = ref board.GetCell(0, 0);
        Assert.AreEqual(1, cell.StackCount);
        Assert.AreEqual(2, cell.UnitLevel);
        Assert.AreEqual(5, cell.OccupantId);
        Assert.IsFalse(MergeResolver.TryMergeAt(board, 0, 0));
        ref var capped = ref board.GetCell(1, 0);
        capped.OccupantId = 6;
        capped.UnitLevel = MergeResolver.MaxLevel;
        capped.StackCount = MergeResolver.MergeStack;
        Assert.IsFalse(MergeResolver.TryMergeAt(board, 1, 0), "max level never merges");
    }
    [Test]
    public void ScannerSweepsAllCompleteStacks()
    {
        var board = new GridBoard();
        foreach (int c in new[] { 0, 4 })
        {
            for (int i = 0; i &lt; 3; i++) board.TryStack(c, 7, 3, 1);
        }
        for (int i = 0; i &lt; 2; i++) board.TryStack(2, 2, 3, 1);
        var scanner = new SameCellThreeMergeScanner(board);
        Assert.AreEqual(2, scanner.ScanAndMerge());
        Assert.AreEqual(2, board.GetCell(0, 7).UnitLevel);
        Assert.AreEqual(2, board.GetCell(4, 7).UnitLevel);
        Assert.AreEqual(2, board.GetCell(2, 2).StackCount, "incomplete stack untouched");
        Assert.AreEqual(0, scanner.ScanAndMerge());
    }
    [Test]
    public void ScanAndLookupAreZeroGc()
    {
        var board = new GridBoard();
        board.TryStack(3, 3, 5, 1);
        board.TryStack(3, 3, 5, 1);
        board.TryStack(3, 3, 5, 1);
```

```
            var scanner = new SameCellThreeMergeScanner(board);
            scanner.ScanAndMerge();
            System.GC.Collect();
            int g0 = System.GC.CollectionCount(0), g1 = System.GC.CollectionCount(1), g2 = System.GC.CollectionCount(2);
            for (int i = 0; i &lt; 1000; i++) scanner.ScanAndMerge();
            Assert.AreEqual(0, System.GC.CollectionCount(0) - g0);
            Assert.AreEqual(0, System.GC.CollectionCount(1) - g1);
            Assert.AreEqual(0, System.GC.CollectionCount(2) - g2);
            System.GC.Collect();
            g0 = System.GC.CollectionCount(0);
            var probe = new Vector2(100f, 100f);
            for (int i = 0; i &lt; 10000; i++) _ = board.WorldToCell(probe);
            Assert.AreEqual(0, System.GC.CollectionCount(0) - g0);
        }
    }
}
// CorePackageTests.cs
using System;
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Core;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P02.Tests
{
    /// &lt;summary&gt;
    /// com.sy.core   NUnit   T001b      ①    +Unsub   ②  v1→v2  +
    /// ③TimeCheated   +Mock   ④        +   CoreCheck batchmode
    /// &lt;/summary&gt;
    [TestFixture]
    public sealed class CorePackageTests
    {
        [SetUp]
        public void SetUp()
        {
            Logger.RemoveSinks();
        }
        [TearDown]
        public void TearDown()
        {
            Logger.RemoveSinks();
        }
        // ── ①      + Unsub   ──
        [Test]
        public void EventBusLoopAndUnsub()
        {
            var bus = new SyEventBus();
            var received = new List&lt;int&gt;();
            void Handler(int e) =&gt; received.Add(e);
            bus.Sub&lt;int&gt;(Handler);
```

```
    bus.Publish(42);
    Assert.AreEqual(1, received.Count, "Sub      ");
    bus.Unsub&lt;int&gt;(Handler);
    bus.Publish(42);
    Assert.AreEqual(1, received.Count, "Unsub           UI   ");
    Assert.AreEqual(0, bus.Count&lt;int&gt;(), "Unsub     0");
    //    Unsub
    int selfFired = 0;
    void SelfUnsub(int e)
    {
        bus.Unsub&lt;int&gt;(SelfUnsub);
        selfFired++;
    }
    bus.Sub&lt;int&gt;(SelfUnsub);
    bus.Publish(7);
    bus.Publish(7);
    Assert.AreEqual(1, selfFired, "   Unsub             ");
    bus.Clear();
}
// ── ② v1→v2 + ──
[Test]
public void SaveMigratesV1ToV2()
{
    var save = new SaveService();
    string slot = "t001b_migration_test";
    PlayerPrefs.DeleteKey(SaveService.Prefix + slot);
    // v1    version       gem
    PlayerPrefs.SetString(SaveService.Prefix + slot, "{\"gem\": 99, \"name\": \"demo\"}");
    string loaded = save.Load(slot);
    var data = MiniJson.Deserialize(loaded) as Dictionary&lt;string, object&gt;;
    Assert.NotNull(data, "        JSON  ");
    Assert.AreEqual(2L, data[SaveService.VersionKey], "     version:2");
    Assert.AreEqual(99L, data["crystal"], "    gem   crystal   ");
    Assert.IsFalse(data.ContainsKey("gem"), "      gem   ");
    Assert.AreEqual("demo", data["name"], "        ");
    string again = save.Load(slot);
    Assert.AreEqual(loaded, again, "       Load   ");
    PlayerPrefs.DeleteKey(SaveService.Prefix + slot);
}
[Test]
public void CloudSyncServerTimestampWinsAndLoserBackedUp()
{
    var endpoint = new FakeEndpoint();
    var save = new SaveService(endpoint);
    string slot = "t001b_sync_test";
    string backupKey = SaveService.BackupPrefix + slot;
    PlayerPrefs.DeleteKey(SaveService.Prefix + slot);
    PlayerPrefs.DeleteKey(backupKey);
    string localPayload = "{\"crystal\": 1}";
    save.Save(slot, localPayload);
```

```
        endpoint.Remote = new CloudSaveRecord
        {
            PayloadJson = "{\"crystal\": 2}",
            ServerTsMs = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds() + 60_000,   //
        };
        var result = save.CloudSync();
        Assert.IsTrue(result.Conflict, "            →    ");
        Assert.AreEqual(backupKey, result.BackupSlot, "           backupSlot");
        Assert.AreEqual("{\"crystal\": 2}", PlayerPrefs.GetString(SaveService.Prefix + slot), "          ");
        StringAssert.Contains("\"crystal\":1", PlayerPrefs.GetString(backupKey), "         backupSlot");
        PlayerPrefs.DeleteKey(SaveService.Prefix + slot);
        PlayerPrefs.DeleteKey(backupKey);
    }
    // ── ③ TimeCheated    + Mock   ──
    [Test]
    public void TimeCheatDetectionThreeCases()
    {
        // SY_MOCK   ServerNow=ClientNow   UtcNow    →  1s
        var mock = new TimeService(null, null, mockMode: true, mockOffsetMs: 0);
        Assert.LessOrEqual(Math.Abs(mock.ServerNowMs() - mock.ClientNowMs()), 1000, "Mock  ServerNow≈ClientNow");
        Assert.IsFalse(mock.TimeCheated(), "  0→    ");
        Assert.AreEqual(TimeSource.Mock, mock.Source, "Mock      Mock");
        var off60 = new TimeService(null, null, true, 60_000);
        Assert.IsFalse(off60.TimeCheated(), "  +60s 60000ms≤120000ms →    ");
        var off121 = new TimeService(null, null, true, 121_000);
        Assert.IsTrue(off121.TimeCheated(), "  +121s 121000ms&amp;gt;120000ms →   ");
        //             →Cloud +
        long fakeServer = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds() - 5_000;
        var cloud = new TimeService(() =&amp;gt; fakeServer, null);
        _ = cloud.ServerNowMs();   //           Source
        Assert.AreEqual(TimeSource.Cloud, cloud.Source, "      →Source=Cloud");
        Assert.IsFalse(cloud.TimeCheated(), "     5s     →    ");
    }
    // ── ④          +    ──
    [Test]
    public void FontSetFallbackWithWarning()
    {
        var warnings = new List&amp;lt;string&amp;gt;();
        Logger.AddSink(new CaptureSink(warnings));
        try
        {
            var locale = new LocaleService(null, null);
            var set = locale.GetFontSet();
            Assert.AreEqual("LegacyRuntime.ttf", set.NormalFont, "          ");
            Assert.AreEqual("LegacyRuntime.ttf", set.BoldFont);
            Assert.AreEqual("LegacyRuntime.ttf", set.NumberFont);
            Assert.Greater(warnings.Count, 0, "         LOCALE  ");
            string text = locale.GetText("t001b_missing_key");
            Assert.AreEqual("t001b_missing_key", text, " key   key  ");
            Assert.Greater(warnings.Count, 1, " key   LOCALE  ");
```

```
            }
            finally
            {
                Logger.RemoveSinks();
            }
        }
        sealed class CaptureSink : Logger.ISink
        {
            readonly List&lt;string&gt; _sink;
            public CaptureSink(List&lt;string&gt; sink) { _sink = sink; }
            public void Write(Logger.Level level, string tag, string message) =&gt; _sink.Add(tag + ":" + message);
        }
        sealed class FakeEndpoint : ICloudSaveEndpoint
        {
            public CloudSaveRecord? Remote;
            public CloudSaveRecord? Fetch(string slot) =&gt; Remote;
            public void Push(string slot, CloudSaveRecord record) { }
        }
    }
}
// PlatformPackageTests.cs
using System;
using System.Linq;
using System.Threading.Tasks;
using NUnit.Framework;
using Sy.Platform;
namespace Sy.P02.Tests
{
    /// &lt;summary&gt;
    /// com.sy.platform NUnit mirror of PlatformCheck (T001c): ad red lines,
    /// grant idempotency, leaderboard paging, bootstrap wiring, support mocks.
    /// &lt;/summary&gt;
    [TestFixture]
    public sealed class PlatformPackageTests
    {
        [Test]
        public void BootstrapResolvesEditorMockSuite()
        {
            var suite = PlatformBootstrap.Create();
            Assert.AreEqual(Channel.Editor, suite.Channel, "Editor domain must resolve the mock suite");
            Assert.NotNull(suite.Platform);
            Assert.NotNull(suite.Ads);
            Assert.NotNull(suite.Grant);
            Assert.NotNull(suite.AntiAddiction);
            Assert.NotNull(suite.Leaderboard);
            Assert.NotNull(suite.OpenData);
            Assert.NotNull(suite.Storage);
            Assert.NotNull(suite.Recording);
            Assert.NotNull(suite.Subscribe);
            Assert.NotNull(suite.Cloud);
```

```
}
[Test]
public void SuiteIdMappingIsStable()
{
    Assert.AreEqual("wx", PlatformBootstrap.SuiteIdFor(Channel.WeChat));
    Assert.AreEqual("douyin", PlatformBootstrap.SuiteIdFor(Channel.Douyin));
    Assert.AreEqual("app", PlatformBootstrap.SuiteIdFor(Channel.App));
    Assert.AreEqual("h5", PlatformBootstrap.SuiteIdFor(Channel.H5));
    Assert.AreEqual("mock", PlatformBootstrap.SuiteIdFor(Channel.Editor));
}
[Test]
public async Task RewardedDailyCapTwelveBlocksThirteenth()
{
    var ads = new MockAdsService(new AdCapPolicy(12), () =&gt; "20260917", null);
    for (int i = 0; i &lt; 12; i++)
    {
        var r = await ads.ShowRewarded("pb");
        Assert.IsTrue(r.Completed, "view " + (i + 1) + " must complete");
    }
    var thirteenth = await ads.ShowRewarded("pb");
    Assert.IsFalse(thirteenth.Completed, "13th view must be blocked");
    Assert.AreEqual("daily_cap", thirteenth.Reason);
    Assert.AreEqual(AdLifecycleType.Fail, ads.Events.Last().Type);
    Assert.AreEqual("daily_cap", ads.Events.Last().Reason);
}
[Test]
public async Task RewardedLifecycleOrderAndAbortPath()
{
    var ads = new MockAdsService(new AdCapPolicy(1), () =&gt; "20260917", null);
    ads.Preload("revive");
    await ads.ShowRewarded("revive");
    CollectionAssert.AreEqual(
        new[]
        {
            AdLifecycleType.Request, AdLifecycleType.Fill,
            AdLifecycleType.Show, AdLifecycleType.Complete
        },
        ads.Events.Select(e =&gt; e.Type).ToArray(), "lifecycle must be request-&gt;fill-&gt;show-&gt;complete")
    var abort = new MockAdsService(new AdCapPolicy(1), () =&gt; "20260917", null) { RewardOutcome = false };
    var aborted = await abort.ShowRewarded("revive");
    Assert.IsFalse(aborted.Completed);
    Assert.AreEqual("aborted", aborted.Reason);
}
[Test]
public void InterstitialCooldownBlocksAndReopens()
{
    float clock = 100f;
    var ads = new MockAdsService(new AdCapPolicy(), () =&gt; "20260917", () =&gt; clock);
    ads.ShowInterstitial("i1");
    int first = Count(ads, AdLifecycleType.Show);
```

```
        clock = 200f;
        ads.ShowInterstitial("i1");
        int second = Count(ads, AdLifecycleType.Show);
        Assert.AreEqual("cooldown", ads.Events.Last().Reason, "inside 180s cooldown must fail");
        clock = 281f;
        ads.ShowInterstitial("i1");
        Assert.AreEqual(1, first, "first show allowed");
        Assert.AreEqual(1, second, "cooldown show blocked");
        Assert.AreEqual(2, Count(ads, AdLifecycleType.Show), "show allowed after cooldown");
    }
    [Test]
    public async Task AdGrantNonceIdempotencyAndDailyCap()
    {
        var grant = new MockAdGrantService(2);
        long ts = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
        Assert.IsTrue((await grant.Verify(Token("n1", ts))).Granted);
        Assert.IsFalse((await grant.Verify(Token("n1", ts))).Granted, "replay must be rejected");
        Assert.IsTrue((await grant.Verify(Token("n2", ts))).Granted);
        Assert.IsFalse((await grant.Verify(Token("n3", ts))).Granted, "daily cap must be enforced");
        Assert.AreEqual(2, grant.UsedNonceCount);
        Assert.IsFalse((await grant.Verify(Token("", ts))).Granted, "empty nonce rejected");
        AdToken Token(string nonce, long tsMs)
            =&gt; new AdToken { PointId = "p", OpenId = "o", Ts = tsMs, Nonce = nonce };
    }
    [Test]
    public async Task LeaderboardPagesThroughCursor()
    {
        var board = new MockLeaderboardService();
        board.Seed("b", 100);
        var p1 = await board.FetchPage("b", TimeSlot.Week, null);
        Assert.AreEqual(20, p1.Items.Count);
        Assert.AreEqual(1, p1.Items[0].Rank);
        Assert.AreEqual(20, p1.Items[19].Rank);
        Assert.IsTrue(p1.HasMore);
        int total = p1.Items.Count;
        string cursor = p1.NextCursor;
        bool more = p1.HasMore;
        int pages = 1;
        while (more &amp;&amp; pages &lt; 10)
        {
            var p = await board.FetchPage("b", TimeSlot.Week, cursor);
            total += p.Items.Count;
            cursor = p.NextCursor;
            more = p.HasMore;
            pages++;
        }
        Assert.AreEqual(100, total, "cursor walk covers all entries");
        Assert.AreEqual(5, pages, "100 entries / 20 per page = 5 pages");
        await board.Submit("b", 999);
        var top = await board.FetchPage("b", TimeSlot.Week, null);
```

```
            Assert.AreEqual(999, top.Items[0].Score);
            Assert.AreEqual(1, top.Items[0].Rank);
        }
        [Test]
        public async Task SupportMocksDegradeSafely()
        {
            var storage = new MockStorage();
            storage.Set("k", "v1");
            Assert.AreEqual("v1", storage.Get("k"));
            storage.Set("k", "v2");
            Assert.AreEqual("v2", storage.Get("k"));
            Assert.IsNull(storage.Get("none"));
            var recording = new MockRecording();
            Assert.IsFalse(recording.IsSupported);
            recording.ShareLastClip("fate_gate_topic");
            Assert.AreEqual("fate_gate_topic", recording.LastTopic);
            var clip = await recording.RecordClipAsync(3);
            Assert.NotNull(clip);
            var subscribe = new MockSubscribe();
            await subscribe.Request(SubscribeType.EnergyFull);
            Assert.AreEqual(SubscribeType.EnergyFull, subscribe.Requests[0]);
            var cloud = new MockCloudService();
            cloud.Register("fn", p =&gt; "ok");
            Assert.AreEqual("ok", await cloud.CallFunction("fn", "{}"));
            Assert.IsNull(await cloud.CallFunction("missing", "{}"));
            Assert.AreEqual(2, cloud.CallCount);
            var open = new MockOpenDataBoard();
            await open.RenderToSharedCanvas("b", new UnityEngine.Rect(0, 0, 1, 1));
            Assert.AreEqual(1, open.RenderCalls.Count);
        }
        static int Count(MockAdsService ads, AdLifecycleType type)
            =&gt; ads.Events.Count(e =&gt; e.Type == type);
    }
}
// UiInputChainTests.cs
using System.Collections;
using NUnit.Framework;
using Sy.P02.Newbie;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem;
using UnityEngine.InputSystem.UI;
using UnityEngine.UI;
using UnityEngine.TestTools;
namespace Sy.P02.PmTests
{
    /// &lt;summary&gt;
    /// X1321 (U262 batch A, 12th doc sec 12 v2.0 input-chain truth law): the
    /// P02 direct-call debt absorption slice. VirtualUserCheck's persona bot
    /// used to fire button.onClick.Invoke() directly (recycle + summon seams)
```

```
/// - the handler path only, skipping raycast, the interactable gate and
/// the EventSystem. This PlayMode domain drives the SAME two flows with
/// SYNTHESIZED touch input through the real UGUI pointer chain
/// (EventSystem + InputSystemUIInputModule + GraphicRaycaster + Button
/// state), so the input-chain truth the old seam could not carry is now
/// proven by injection instead of hand-waved to L2/L3. Rig mirrors the
/// editor BuildRig verbatim (WorldSpace 1080x1920 canvas + ortho camera
/// + teaching BattleSim + BattleView) plus the P08 X1297 rig additions;
/// the world-space rig keeps every seat tappable at any Game View size.
/// &lt;/summary&gt;
[TestFixture]
public sealed class UiInputChainTests : InputTestFixture
{
    GameObject _root;
    Canvas _canvas;
    Camera _cam;
    BattleSim _sim;
    BattleView _view;
    Button _summonBtn;
    Button _recycleBtn;
    int _summonEvents;
    int _recycleEvents;
    int _recycleDone;
    const int UiLayer = 5;
    public override void Setup()
    {
        base.Setup();
        // NUnit reuses one fixture instance across the tests in this
        // fixture, so the event ledgers must be zeroed per test (the
        // N3 roam touches both counters and exposed the shared state).
        _summonEvents = 0;
        _recycleEvents = 0;
        _recycleDone = 0;
        _root = new GameObject("UiInputChainRig");
        var esGo = new GameObject("TestEventSystem");
        esGo.transform.SetParent(_root.transform, false);
        esGo.AddComponent&lt;EventSystem&gt;();
        esGo.AddComponent&lt;InputSystemUIInputModule&gt;();
        var camGo = new GameObject("Cam");
        camGo.transform.SetParent(_root.transform, false);
        _cam = camGo.AddComponent&lt;Camera&gt;();
        _cam.orthographic = true;
        _cam.orthographicSize = 960f;             // half the 1920 design height
        _cam.aspect = 1080f / 1920f;
        _cam.transform.position = new Vector3(0f, 0f, -100f);
        _cam.farClipPlane = 2000f;
        _cam.cullingMask = 1 &lt;&lt; UiLayer;
        var canvasGo = new GameObject("RigCanvas");
        canvasGo.transform.SetParent(_root.transform, false);
        _canvas = canvasGo.AddComponent&lt;Canvas&gt;();
```

```
    _canvas.renderMode = RenderMode.WorldSpace;
    _canvas.worldCamera = _cam;
    ((RectTransform)canvasGo.transform).sizeDelta = new Vector2(1080f, 1920f);
    canvasGo.AddComponent&lt;GraphicRaycaster&gt;();
    _sim = new BattleSim(seed: 20260916, teachingFloor: true);
    var viewGo = new GameObject("BattleView");
    viewGo.transform.SetParent(_root.transform, false);
    _view = viewGo.AddComponent&lt;BattleView&gt;();
    _view.Initialize(_canvas, _sim);
    SetLayer(_root, UiLayer);
    // Host glue in the editor-rig shape (VirtualUserCheck parity):
    // view event -&gt; handler -&gt; sim op.
    _view.SummonRequested += () =&gt;
    {
        _summonEvents++;
        _sim.TrySummon(out _, out _);
    };
    _view.RecycleRequested += () =&gt;
    {
        _recycleEvents++;
        var unit = _sim.FindRecyclableUnit();
        if (unit != null &amp;&amp; _sim.TryRecycleUnit(unit, out _)) _recycleDone++;
    };
    _summonBtn = FindButton("SummonButton");
    _recycleBtn = FindButton("RecycleButton");
    _sim.StartBattle();
}
public override void TearDown()
{
    if (_root != null) Object.DestroyImmediate(_root);
    _root = null;
    _canvas = null;
    _cam = null;
    _sim = null;
    _view = null;
    _summonBtn = null;
    _recycleBtn = null;
    base.TearDown();
}
// ——— flow 1: the summon seam (VirtualUserCheck Tap) through the
// real pointer chain - a synthesized touch at the live button's
// runtime screen point must land the summon exactly like a finger.
// ———
[UnityTest]
public IEnumerator SummonFlow_InjectedTouch_WalksRealPointerChain()
{
    Assert.NotNull(_summonBtn, "the view must build the summon seat");
    Assert.IsTrue(_summonBtn.interactable, "the summon seat ships live");
    Assert.Greater(_sim.FreeSummons, 0, "teaching floor grants free summons");
    int units = _sim.Units.Count;
```

```
    int crystals = _sim.Crystals;
    yield return TouchTap(_summonBtn);
    Assert.AreEqual(1, _summonEvents, "the injected touch reached the view event");
    Assert.AreEqual(units + 1, _sim.Units.Count,
        "the tap walked the whole chain: event -&gt; handler -&gt; sim op (a unit landed)");
    Assert.AreEqual(crystals, _sim.Crystals, "a free summon never spends crystals");
}
// ——— flow 2: the recycle seam (VirtualUserCheck novice persona)
// through the real pointer chain - including the interactable gate
// the old onClick.Invoke() seam bypassed: with no white unit on the
// board the seat must swallow the pointer, and the exact opposite
// once a recyclable unit exists.
// ———
[UnityTest]
public IEnumerator RecycleFlow_InjectedTouch_RespectsTheInteractableGate()
{
    Assert.NotNull(_recycleBtn, "the view must build the recycle seat");
    // Stage a deterministic white unit the way the editor gates do.
    _sim.InjectUnitForTests(0, 1, 2, 4);
    _view.SyncView(_sim);
    Assert.IsTrue(_recycleBtn.interactable, "a recyclable unit lights the seat");
    int units = _sim.Units.Count;
    int crystals = _sim.Crystals;
    yield return TouchTap(_recycleBtn);
    Assert.AreEqual(1, _recycleEvents, "the live seat routed the tap to the view event");
    Assert.AreEqual(1, _recycleDone, "the handler recycled the staged white unit");
    Assert.AreEqual(units - 1, _sim.Units.Count, "the recycled unit left the board");
    Assert.Greater(_sim.Crystals, crystals, "the refund landed in the wallet");
    // The gate face the old seam could never prove: with the white
    // unit gone the seat goes dark and the same finger tap at the
    // same screen point now hits nothing - the pointer chain swallows
    // it before the onClick delegate is ever reached.
    _view.SyncView(_sim);
    Assert.IsFalse(_recycleBtn.interactable, "no recyclable unit dims the seat");
    yield return TouchTap(_recycleBtn);
    Assert.AreEqual(1, _recycleEvents, "the dead seat swallowed the injected tap");
    Assert.AreEqual(1, _recycleDone, "no recycle happened through the dark seat");
}
// ——— N3 (X1589 design freeze sec 2, slice c): the random-roam bot
// law. A seeded roam - not a script: blind points across the whole
// face (placeholder bands, dead corners, off-face screen) interleaved
// with pokes at the discovered seats, every one through the real
// pointer chain. Faces: zero softlock (every poke lands inside the
// frame guard and no unexpected log survives); first-summon reachable
// (the opening beat taps the live free-triple seat and the summon
// lands at t=0); progress non-zero (waves/kills/crystals move under
// the roaming finger). The harness only plays the launcher's
// star-pick beat when the sim pauses - this rig wires no star panel;
// sim values are never touched.
// ———
```

```
[UnityTest]
public IEnumerator N3_RandomRoamBot_ZeroSoftlock_FirstSummonReachable_ProgressNonZero()
{
    const int TotalPokes = 72;   // freeze law: &gt;= 60 pokes on the rig
    const float Dt = 0.05f;      // fixed sim step, editor-check parity
    var rng = new System.Random(20261001);
    Assert.Greater(_sim.FreeSummons, 0, "the free triple is live before the first beat");
    int crystals0 = _sim.Crystals;
    int blindPokes = 0;
    int landed = 0;   // loop counter - proves the roam never wedged
    // The 1080x1920 face rect in screen space, so in-face blind pokes
    // can land on placeholder bands (HUD strips, board gaps) too.
    var faceMin = RectTransformUtility.WorldToScreenPoint(_cam, new Vector3(-540f, -960f, 0f));
    var faceMax = RectTransformUtility.WorldToScreenPoint(_cam, new Vector3(540f, 960f, 0f));
    for (int poke = 0; poke &lt; TotalPokes; poke++)
    {
        Vector2 point;
        if (poke == 0)
        {
            // Opening beat: the live first-summon seat is inside the
            // roam's poke domain by construction.
            point = SeatPoint(_summonBtn);
        }
        else if (poke % 3 == 0)
        {
            // Discovered-seat roam - live or dead, the interactable
            // gate decides what the finger does, not the bot.
            point = SeatPoint(rng.Next(2) == 0 ? _summonBtn : _recycleBtn);
        }
        else
        {
            blindPokes++;
            bool inFace = faceMax.x &gt; faceMin.x &amp;&amp; faceMax.y &gt; faceMin.y &amp;&amp; (rng.Next() &a
            point = inFace
                ? new Vector2(faceMin.x + (float)rng.NextDouble() * (faceMax.x - faceMin.x),
                              faceMin.y + (float)rng.NextDouble() * (faceMax.y - faceMin.y))
                : new Vector2(rng.Next(Mathf.Max(1, Screen.width)), rng.Next(Mathf.Max(1, Screen.height)));
        }
        int eventsBefore = _summonEvents;
        yield return TouchTapAt(point);
        if (poke == 0)
        {
            Assert.AreEqual(1, _summonEvents - eventsBefore,
                "opening beat: the live seat routed the finger to the view event");
            Assert.AreEqual(1, _sim.SummonCount,
                "the free summon landed through the real pointer chain at t=0");
        }
        // The battle clock runs under the roaming finger (fixed step
        // keeps the roam deterministic across machines).
        for (int f = 0; f &lt; 8; f++)
```

```
            {
                if (_sim.State == BattleSim.BattleState.StarCardPause) _sim.PickStarCard(0);
                _sim.Tick(Dt);
                yield return null;
            }
            _view.SyncView(_sim);
            landed++;
        }
        Assert.AreEqual(TotalPokes, landed,
            "zero softlock: the roam completed every poke inside the frame guard");
        Assert.GreaterOrEqual(blindPokes, 30,
            "blind pokes carried at least a third of the roam (got " + blindPokes + ")");
        bool progressed = _sim.Kills &gt; 0 || _sim.Wave &gt;= 2 || _sim.Crystals != crystals0;
        Assert.IsTrue(progressed, "progress face non-zero: wave=" + _sim.Wave
            + " kills=" + _sim.Kills + " crystals " + crystals0 + "-&gt;" + _sim.Crystals);
        Assert.Greater(_sim.SummonCount, 0, "the summon book moved under the roaming finger");
        LogAssert.NoUnexpectedReceived();
    }
    // ——— rig helpers ———
    Button FindButton(string name)
    {
        foreach (var b in _canvas.GetComponentsInChildren&lt;Button&gt;(true))
            if (b.gameObject.name == name) return b;
        return null;
    }
    /// &lt;summary&gt;Runtime screen point of a seat's center (world-space rig).&lt;/summary&gt;
    Vector2 SeatPoint(Button target)
    {
        var rt = (RectTransform)target.transform;
        return RectTransformUtility.WorldToScreenPoint(_cam, rt.position);
    }
    /// &lt;summary&gt;One synthesized finger tap at an arbitrary screen point -
    /// the roam variant of TouchTap (down, one frame, up, two settle frames).&lt;/summary&gt;
    IEnumerator TouchTapAt(Vector2 screenPos)
    {
        Assert.Greater(Screen.width, 0, "player has no screen face in this run");
        Assert.Greater(Screen.height, 0, "player has no screen face in this run");
        BeginTouch(1, screenPos);
        yield return null;
        EndTouch(1, screenPos);
        yield return null;
        yield return null;
    }
    static void SetLayer(GameObject root, int layer)
    {
        root.layer = layer;
        foreach (Transform tr in root.GetComponentsInChildren&lt;Transform&gt;(true))
            tr.gameObject.layer = layer;
    }
    /// &lt;summary&gt;One synthesized finger tap at the target's runtime screen
```

```
        /// point: touch down, one frame, touch up, then two frames so the
        /// pointer chain (module update -&gt; EventSystem -&gt; onClick) settles
        /// before the caller asserts.&lt;/summary&gt;
        IEnumerator TouchTap(Button target)
        {
            var rt = (RectTransform)target.transform;
            var screenPos = RectTransformUtility.WorldToScreenPoint(_cam, rt.position);
            Assert.Greater(Screen.width, 0, "player has no screen face in this run");
            Assert.Greater(Screen.height, 0, "player has no screen face in this run");
            BeginTouch(1, screenPos);
            yield return null;
            EndTouch(1, screenPos);
            yield return null;
            yield return null;
        }
    }
}
// TelemetryPackageTests.cs
using System;
using System.Collections.Generic;
using System.IO;
using System.Threading.Tasks;
using NUnit.Framework;
using Sy.Core;
using Sy.Platform;
using Sy.Telemetry;
using UnityEngine;
namespace Sy.P02.Tests
{
    /// &lt;summary&gt;
    /// com.sy.telemetry NUnit mirror of TelemetryCheck (T001e): schema load from the
    /// real cfg_event table, event classes, validation marks, analytics sink, batching.
    /// &lt;/summary&gt;
    [TestFixture]
    public sealed class TelemetryPackageTests
    {
        sealed class CaptureSink : TelemetryService.ISink
        {
            public readonly List&lt;KeyValuePair&lt;string, Dictionary&lt;string, string&gt;&gt;&gt; Events = new();
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
                =&gt; Events.Add(new KeyValuePair&lt;string, Dictionary&lt;string, string&gt;&gt;(evtId, fields));
        }
        static string RepoRoot()
            =&gt; Path.GetFullPath(Path.Combine(Application.dataPath, "..", "..", ".."));
        static TelemetryService RealSchemaService()
        {
            var svc = new TelemetryService();
            string json = File.ReadAllText(Path.Combine(RepoRoot(), "Design", "configs", "P02", "cfg_event.json"));
            Assert.IsTrue(svc.LoadSchemaFromJson(json), "cfg_event.json must load");
            Assert.AreEqual(13, svc.SchemaEventCount);   // T023-B: evt_run joined; evt_unit_ability restored (X106 past
```

```
    return svc;
}
[Test]
public void SchemaLoadsFromRealConfigTable()
{
    Assert.IsFalse(new TelemetryService().LoadSchemaFromJson("{broken"));
    _ = RealSchemaService();
}
[Test]
public void FiveEventClassesValidateClean()
{
    var svc = RealSchemaService();
    var capture = new CaptureSink();
    svc.AddSink(capture);
    for (int step = 1; step &lt;= 5; step++)
        svc.Track(EventIds.NewbieStep, new Dictionary&lt;string, string&gt; { { "step", step.ToString() } });
    CollectionAssert.AreEqual(new[] { "1", "2", "3", "4", "5" },
        new List&lt;string&gt;(new[] {
            capture.Events[0].Value["step"], capture.Events[1].Value["step"],
            capture.Events[2].Value["step"], capture.Events[3].Value["step"],
            capture.Events[4].Value["step"] }), "newbie step order 1..5");
    Assert.AreEqual(0, svc.LastWarnings.Count);
    foreach (var state in new[] { "request", "fill", "show", "complete", "fail" })
    {
        var fields = new Dictionary&lt;string, string&gt; { { "state", state }, { "pointId", "p1" } };
        if (state == "fail") fields["reason"] = "no_fill";
        svc.Track(EventIds.Ad, fields);
    }
    Assert.AreEqual(0, svc.LastWarnings.Count, "ad funnel validates clean");
    foreach (var stage in new[] { "send", "click", "open" })
        svc.Track(EventIds.Share, new Dictionary&lt;string, string&gt; { { "stage", stage }, { "rewardBoth", "tr
    Assert.AreEqual(0, svc.LastWarnings.Count, "share stages validate clean");
    svc.Track(EventIds.WaveFail, new Dictionary&lt;string, string&gt; { { "mapId", "m1" }, { "waveId", "10" } })
    svc.Track(EventIds.KpiDaily, new Dictionary&lt;string, string&gt; { { "date", "2026-09-17" } });
    Assert.AreEqual(0, svc.LastWarnings.Count, "wave fail and kpi daily validate clean");
    Assert.AreEqual(13, capture.Events.Count);
}
[Test]
public void ValidationMarksUnknownMissingAndBadType()
{
    var svc = RealSchemaService();
    var capture = new CaptureSink();
    svc.AddSink(capture);
    svc.Track("evt_unknown", new Dictionary&lt;string, string&gt;());
    Assert.AreEqual(1, capture.Events.Count, "unknown still written");
    Assert.AreEqual(1, svc.LastWarnings.Count);
    svc.Track(EventIds.WaveFail, new Dictionary&lt;string, string&gt; { { "mapId", "m1" } });
    Assert.AreEqual("waveId", capture.Events[1].Value["__missing"]);
    Assert.AreEqual(1, svc.LastWarnings.Count);
    svc.Track(EventIds.WaveFail, new Dictionary&lt;string, string&gt; { { "mapId", "m1" }, { "waveId", "xyz" } }
```

```
            Assert.AreEqual("waveId", capture.Events[2].Value["__badtype"]);
            Assert.AreEqual(1, svc.LastWarnings.Count);
        }
        [Test]
        public void AnalyticsSinkReportsPerEvent()
        {
            var reported = new List&lt;KeyValuePair&lt;string, string&gt;&gt;();
            var sink = new WxAnalyticsSink(new RecordingReporter(reported));
            sink.Write(EventIds.NewbieStep, new Dictionary&lt;string, string&gt; { { "step", "2" } });
            Assert.AreEqual(1, reported.Count);
            Assert.AreEqual(EventIds.NewbieStep, reported[0].Key);
            var parsed = MiniJson.Deserialize(reported[0].Value) as Dictionary&lt;string, object&gt;;
            Assert.NotNull(parsed);
            Assert.AreEqual("2", parsed["step"]);
        }
        sealed class RecordingReporter : IAnalyticsReporter
        {
            readonly List&lt;KeyValuePair&lt;string, string&gt;&gt; _sink;
            public RecordingReporter(List&lt;KeyValuePair&lt;string, string&gt;&gt; sink) { _sink = sink; }
            public void Report(string evtId, string fieldsJson)
                =&gt; _sink.Add(new KeyValuePair&lt;string, string&gt;(evtId, fieldsJson));
        }
        [Test]
        public async Task TcbBatchSinkFlushesInChunks()
        {
            var sink = new TcbBatchSink(50);
            for (int i = 0; i &lt; 120; i++)
                sink.Write(EventIds.NewbieStep, new Dictionary&lt;string, string&gt; { { "step", "1" } });
            Assert.AreEqual(120, sink.BufferedCount);
            Assert.IsTrue(sink.NeedsFlush);
            var calls = new List&lt;string&gt;();
            var cloud = new CallbackCloud(calls.Add);
            int sent = await sink.FlushAsync(cloud);
            Assert.AreEqual(120, sent);
            Assert.AreEqual(0, sink.BufferedCount);
            Assert.AreEqual(3, calls.Count, "flush batches 50/50/20");
            Assert.AreEqual(0, await sink.FlushAsync(cloud), "empty flush sends nothing");
        }
        sealed class CallbackCloud : ICloudService
        {
            readonly Action&lt;string&gt; _onCall;
            public CallbackCloud(Action&lt;string&gt; onCall) { _onCall = onCall; }
            public Task&lt;string&gt; CallFunction(string functionName, string payloadJson)
            {
                _onCall(functionName);
                return Task.FromResult("{\"ok\":true}");
            }
        }
    }
}
```